

The Silicon Ledger

VOL. 1, NO. 090

2026-03-30

FEATURES

Handling generic Http errors across your Android app

Miguel Juárez López · Yammer Engineering

Using Otto and Retrofit to log out users when receiving 401 error responses from the API

I was recently working on an Android project where we needed to log out the current user whenever you encountered a 401 Unauthorized HTTP response from our API.

I am a fan of all the Android libraries from square , so for this post I'll be using the REST client Retrofit in combination with the event bus Otto to achieve a centralized elegant way to do this.

Although all this should have been pretty straightforward there are a couple of gotchas to be aware of, mostly regarding Otto limitations so let's get to it!

0. Create an event object for Otto

```
public class HttpUnauthorizedEvent { }
```

The way Otto works is by creating a contract between publishers and subscribers based on the type of the object used as a parameter. In this case we will use the `HttpUnauthorizedEvent` object to signal that a 401 error has happened in our application.

An instance of any class may be published on the bus and it will only be dispatched to subscribers for that type.

1. Create a custom `ErrorHandler` for Retrofit and post events for intercepted 401 errors using Otto event bus from there

```
public class CustomErrorHandler implements ErrorHandler {
    private static final Handler MAIN_LOOPER_HANDLER = new Handler(Looper.getMainLooper());
    private final Bus eventBus;

    public CustomErrorHandler(Bus eventBus) {
        this.eventBus = eventBus;
    }

    @Override public Throwable handleError(RetrofitError error) {
        Response r = error.getResponse();
        if (r != null && r.getStatus() == HttpURLConnection.HTTP_UNAUTHORIZED) {
            // we need to make sure we post in the UI thread
            MAIN_LOOPER_HANDLER.post(new Runnable() {
                @Override public void run() {
                    eventBus.post(new HttpUnauthorizedEvent());
                }
            });
        }
        return error;
    }
}
```

One thing to notice is that we're making sure we're posting in the UI thread, this is mainly for two reasons:

By default, all interaction with an instance is confined to the main thread.

If you try to post from a different thread Otto will throw an exception.

We actually want to react to these errors in the UI thread, which is where we will be logging out the user (e.g. in an `Activity`).

2. Configure the `CustomErrorHandler` to be used by Retrofit `RestAdapter`

```
// Obtain Bus EventBus from constructor or singleton injection
Bus eventBus = new BusProvider().get();
// Set retrofit ErrorHandler to use our CustomErrorHandler and
// pass the eventBus to it
RestAdapter restAdapter = new RestAdapter.Builder().setServer("API_URL").setErrorHandler(new CustomErrorHandler(eventBus)).build();
// create and use your retrofit REST client
RestApiClient api = restAdapter.create(RestApiClient.class);
api.MethodCall(...);
// If a 401 error response happens here the error handler will automatically notify all subscribers
```

3. Subscribe for errors on an inner-class of your (base) `Activity`

For my specific case I wanted to put the subscription and log-out logic in a base `Activity` from which all my app `Activities` were inheriting. This is currently impossible given that Otto doesn't support subscribing on base classes.

Otto will not traverse the class hierarchy and add methods from base classes or interfaces that are annotated. This is an explicit design decision to improve performance of the library as well as keep your code simple and unambiguous.

As a workaround you can put your `@Subscribe` method on your base `Activity`'s nested class and it will work as a charm.

```
public abstract class BaseActivity ... {
    private class AuthFailureHandler {
```

```
        @Subscribe
```

```
        public void onAuthFailure(HttpUnauthorizedEvent event) {
            BaseActivity.this.logoutCurrentUser();
        }
    }
}
```

```
// Obtain same Singleton EventBus
```

```
private Bus eventBus = new BusProvider().get();
```

```
private AuthFailureHandler authFailureHandler;
@Override
```

```
protected void onResume() {
```

```
    super.onResume();
```

```
    authFailureHandler = new AuthFailureHandler();
```

```
// "In order to receive events, a class instance needs to
```

```
// register with the bus."
```

```
eventBus.register(authFailureHandler);
```

I Failed to Implement the Diameter of a Binary Tree in a Coding Interview

Daniel Doubrovkine · Daniel Doubrovkine (dBlock)

Six months ago I failed a basic coding interview at a FAANG. Yes, I was a Principal Engineer, and yes, I was paid absurd amounts of money, yet I couldn't implement a diameter of a binary tree as a "warm up" exercise, 10 lines of code.

The interviewer was very nice about it, and was equally surprised. You see, I've been coding for 35 years, and I am not "rusty" at it, which is a typical excuse for senior ICs bombing LeetCode interviews. At that time I was writing code every day. I simply froze, and couldn't do it. All I could think of was "why the hell am I here doing this to myself?". I couldn't turn my brain around, apologized, cut the interview short, and, while I did ace several coding interviews at other companies for IC roles, went back to management for which I am, obviously, more qualified.

Did you just bomb a coding interview? Here's some practical advice of how to deal with it.

First, keep your head high and do not let this negative experience get to you. It feels awful, but unless you are blogging about your failure to implement the diameter of a binary tree, nobody other than you and the interviewer will ever know. Most likely the interviewer was also a half asleep drone deep inside the borg, administering a coding puzzle every day to the next sucker, so they will forget about you the next day.

I see a lot of junior Engineers affected by failing at these. If you are recently out of college, it's helpful to think of these interviews as a school test, such as an SAT. I never had to take SATs (my children did), but most European schools have oral exams where you get a ticket from a jar with the problem statement. I had slept through too many alarms to attend an 8 AM physics class around 1996 because of a grueling late night partying schedule, and only really knew maybe half the material. Luckily 2/3 questions in my ticket overlapped with those classes I did attend, and so I did well.

If you don't like school and prefer video games, think of these interviews as Crossy Road. You need some skill, but mostly you just need some luck. There's a set of problems you know how to solve, and a set you don't. Practicing is important, but nothing helps like the fact that I was asked to implement an LRU Cache twice in the last decade in coding interviews. You also need to only cross one road. Failed? Try to cross half a mile down.

If you are a junior IC, remember that these LeetCode-style interviews are dumb and there's nothing you can do about it. Fifteen years ago I wrote a viral blog post entitled GitHub is Your New Resume. Nothing changed since. In hiring from open-source at my last job at AWS I tried to swap the coding interview requirement for our team's hiring for an offline evaluation of one's open-source code. Because, you know, these are active open-source contributors and we are sure they can code! Naturally, I failed to change anything. I

was told that we needed a consistent interviewing process to evaluate everyone the same way. And so we consistently hired people that could code on a whiteboard, a complete self-own.

If you are a senior IC, remember that you are not paid to write diameters of binary trees. You don't really want to work at the company that gives you that type of problems at the interview. The job of a Principal Engineer is to have resounding impact on a time scale of 1 year. You pick the most important unsolved problem and you solve it, long term. This may include prototyping a meaningful piece of software that changes the course of a business, or gaining support from senior leadership for a directional alignment for hundreds of people. It could be enabling teams in ways they couldn't think of, and so many other ways to have the kind of impact that people with decades of experience tend to occasionally produce. None of it is the kind of code a junior Engineer or an AI can crank out in seconds in 2025.

Speaking of AI, we have one chance to get rid of these stupid interviews now. It has become very clear that only companies that embrace AI will survive, so the best way to evaluate a candidate is to let them use AI, which gets rid of having to implement solutions to these problems in coding interviews.

Good luck with your interviews!

I Failed to Implement the Diameter of a Binary Tree in a Coding Interview was originally published by Daniel Doubrovkine at code.dblock.org | tech blog on June 21, 2025.

Seven Schools, Four Cities, and Three Countries Later

Mannie Tagarira · Yammer Engineering

A Conversation About Diversity

Let's examine what lies across country borders.

Growing up, I wondered what lay across country borders. What makes their country special compared to any other? Why is their food so different from ours? I was always curious about what people from other nations were like, so I dreamed of visiting distant lands. My mother's stories, stories I vividly remember, of her travels to neighboring countries encouraged those dreams; I had so much hope that one day she'd see the world. I've held on to her stories, sometimes returning to them as if they were receipts for experiences that I too might have.

My Last Few Months of Schooling in Zimbabwe

I was in a class of kids that, for the most part, grew up in the same society. As I remember it, there was one Pakistani girl — Raima. She hardly spoke to anyone, it seemed. She kept to herself. Whenever I'd walk over to say 'hi' and make small talk (maybe flirt a little), she would clam up. 'Damn, this girl is shy', I'd think after any attempt to befriend her. Now I can't help but wonder, 'Was it really shyness or maybe something else?'

In 2004, I relocated to Manchester, England. This was a move that would ultimately change me. At the time, my young mind hadn't yet fathomed the significance of the transition: opportunity with responsibility, a gift and a curse.

Excited but Terribly Confused

The experience of being in a new country with unfamiliar customs and a different education system was exhilarating and baffling at the same time. I struggled to fit in and found myself in a tug-of-war between teachers who liked me because they saw my potential and teachers who felt that I was shown favoritism for being in 'the minority group'. I can't explain why, but I just couldn't fit in, even with kids who had the same skin color as mine. I'd always thought of myself as a friendly and sociable individual. What was I doing wrong? It didn't take too long for me to realize how Raima felt.

In hindsight, it was neither a problem of nerves nor a matter of what we were possibly doing wrong. It was simpler than that. This was about our differences—having different backgrounds and customs from one another. Humans are predisposed towards comfort. We seek out individuals who most resemble our own selves, looking for a resemblance that, at times, transcends physical appearance. As such, I found myself becoming really good friends with two fellas—Aamir from Pakistan and Bashir from Somalia—who shared passion and interests similar to mine. We bonded over the realities of being outcasts.

I had to decide where I would complete my A-Levels. Aamir and I both made our decisions somewhat based on emotion, seeking out an environment that was most familiar. At the school I settled into, there were more people who shared my upbringing and looked more like me, and I found myself becoming good friends with a collective of talented and resourceful individuals who grew up in southern Africa. Over the next couple of years, we would geek out over music and dream of one day owning our own businesses in hopes of going back to Africa and improving the economy. You know, 'making it big', whatever that meant at the time. Even though these aspirations kept me connected to my new group of friends, I could never really imagine going 'home'. Would I be able to re-integrate into that society? I had already started to change. I had gotten so used to attention; most of it wasn't a result of my achievements, but rather just being myself.

By graduation day, I had effectively spent 5 of the preceding 7 years training for an industry that may not accept the various stereotypes I conformed to. I was a twenty-something-year-old black male with a love for basketball, hip hop and everything that came along with the two. Even my hair and dress sense reflected my pastimes—something I had to change in order to fit in with the professional crowd.

London, August of 2011

I joined an investment banking firm. Within the first few days around the office, I noticed a popular topic of discussion: diversity .

Why is this "diversity thing" so important? I quickly came to understand that the concept is intended to help people feel empowered through acknowledgement of their differences. Let me explain: Working with people of similar backgrounds allows for a certain level of confidence and comfort. There's that 'comfort' word, again... At the least, I think we'd all agree that communication is somewhat effortless if we speak the same language (both literally and metaphorically). Makes sense, right?

Now 7 schools, 3 cities and 2 countries in, I was ready to take on the world, diverse or not. I felt like everything I had ever learned was in preparation for that moment, the moment my career officially began. You see, before I joined the firm, I had already met a lot of intelligent individuals, which were rather humbling experiences. Some of the most unassuming of characters I crossed paths with had ideas, really smart ones, that bordered insanity!

I had no misconceptions about my level of intelligence coming into this... or going into the corporate world, rather. Being part of that large corporation was just unnerving. I was indeed a small fish in a pond filled with other well-decorated fish. How does one even stand out? Heck, how did I even blend in?

A Bad Case of Imposter Syndrome

Adventures in Benchmarking - Performance Golf

Matt Warren (.NET)

Recently Nick Craver one of the developers at Stack Overflow has been tweeting snippets of code from their source, the other week the following code was posted:

A daily screenshot from the Stack Overflow codebase (checking strings for tokens without allocations). #Stack-Code pic.twitter.com/sDPqviHgD0

— Nick Craver (@Nick_Craver) April 20, 2016

This code is an optimised version of what you would normally write, specifically written to ensure that it doesn't allocate memory. Previously Stack Overflow have encountered issues with large pauses caused by the .NET GC, so it appears that where appropriate, they make a concerted effort to write code that doesn't needlessly allocate.

I also have to give Nick credit for making me aware of the term "Performance Golf", I've heard of Code Golf, but not the Performance version.

Aside: If you want to see the full discussion and the code for all the different entries, take a look at this gist. Also for a really in-depth explanation of what the fastest version is actually doing, I really recommend checking out Kevin Montrose's blog post "An Optimisation Exercise", there's some very cool tricks in there, although by this point he is basically writing C/C++ code rather than anything you would recognise as C#!

In this post I'm not going to concentrate too much on this particular benchmark, but instead I'm going to use it as an example of what I believe a good benchmarking library should provide for you. Full disclaimer, I'm one of the authors of BenchmarkDotNet, so I admit I might be biased!

I think that a good benchmarking tool should offer the following features:

Diagnose what is going on

Consistent, Reliable and Clear Results

By using BenchmarkDotNet, or indeed any benchmarking tool, you can just get on with the business of actually writing the benchmark and not worry about any of the mechanics of accurately measuring the code. This is important because often when someone has posted an optimisation and accompanying benchmark on Stack Overflow, several of the comments then point out why their measurements are inaccurate or plain wrong.

In the case of BenchmarkDotNet, it's as simple as adding a [Benchmark] attribute to the methods that you want to benchmark and then a few lines of code to launch the run:

```
[ Benchmark ( Baseline = true ) ] public bool StringSplit ()
{ var tokens = Value . Split ( delimiter ); foreach ( var token
in tokens ) { if ( token == Match ) return true ; } return false ; }
```

```
static void Main ( string [] args ) { var summary = Bench-
markRunner . Run (); }
```

It also offers a few more tools for advanced scenarios, for instance you can decorate a field/property with the [Params] attribute like so:

```
[ Params ( "Foo;Bar" , "Foo;FooBar;Whatever" ,
"Bar;bla;foo" , "bla;foo;Bar" , "foo;Bar;Bla" ,
"foo;FooBar;Bla" , "Bar1;Bar2;Bar3;Bar4;Bar" ,
"Bar1;Bar2;Bar3;Bar4;NoMatch" , "Foo;FooBar;Whatever" ,
"Some;Other;Really;Interesting;Tokens" ) ] public string
Value { get ; set ; }
```

and then each benchmark will be run multiples times, with Value set to the different strings. This gives you a really easy way of trying out benchmarks across different inputs. For instance some methods were consistently fast, whereas others performed badly on inputs that were a worse-case scenario for them.

Diagnose what is going on

If you state that the aim of optimising your code is to "check a string for tokens, without allocations", you would really like to be able to prove if that is true or not. I've previously written about how BenchmarkDotNet can give you this information and in this case we get the following results (click for full-size image):

So you can see that the ContainTokenFransBouma benchmark isn't allocation free, which in the scenario is a problem.

Consistent, Reliable and Clear Results

Another important aspect is that you should be able to rely on the results. Part of this is trusting the tool and hopefully people will come to trust BenchmarkDotNet over time.

Also you should be able to get clear results, so in as well as providing a text-based result table that you can easily paste into a GitHub issue or Stack Overflow answer, BenchmarkDotNet will provide several graphs using the R statistics and graphing library. Sometimes a wall of text isn't the easiest thing to interpret, but colourful graphs can help (click for full image).

Here we can see that the original ContainsToken code is "slower" in some scenarios (although it's worth pointing out that the Y-axis is in nanoseconds).

Would I recommend writing code like any of these optimisations for normal day-to-day scenarios? No.

Without exception the optimised versions of the code are less readable, harder to debug and probably contain more errors. Certainly, by the time you get to the fastest version

For instance some methods were consistently fast, whereas other performed badly on inputs that were a worse-case scenario for them.

Matt Warren (.NET)

CTCSS fingerprinting: a method for transmitter identification

Oona Räisänen · Oona Raisenen (windytan)

Identifying unknown radio transmitters by their signals is called radio fingerprinting. It is usually based on rise-time signatures, i.e. characteristic differences in how the transmitter frequency fluctuates at carrier power-up. Here, instead, I investigate the fingerprintability of another feature in hand-held FM transceivers, known as CTCSS or Continuous Tone-Coded Squelch System.

Motivation & data

I came across a long, losslessly compressed recording of some walkie-talkie chatter and wanted to know more about it, things like the number of participants and who's talking with who. I started writing a transcript – a fun pastime – but some voices sounded so similar I wondered if there was a way to tell them apart automatically.

The file comprises several thousand short transmissions as FM demodulated audio lowpass filtered at 4500 Hz. Signal quality is variable; most transmissions are crisp and clear but some are buried under noise. Passages with no signal are squelched to zero.

I considered several potentially fingerprintable features, many of them unrealistic:

Carrier power-up; but many transmissions were missing the very beginning because of squelch

Voice identification; but it would probably require pretty sophisticated algorithms (too difficult!) and longer samples

Mean audio power; but it's not consistent enough, as it depends on text, tone of voice, etc.

Maximum audio power; but it's too sensitive to peaks in FM noise

I then noticed all transmissions had a very low tone at 88.5 Hz. It turned out to be CTCSS, an inaudible signal that enables handsets to silence unwanted transmissions on the same channel. This gave me an idea inspired by mains frequency analysis : Could this tone be measured to reveal minute differences in crystal frequencies and modulation depths? Also, knowing that these were recorded using a cheap DVB-T USB stick – would it have a stable enough oscillator to produce consistent measurements?

I used the liquid-dsp library for signal processing. It has several methods for measuring frequencies. I decided to use a phase-locked loop, or PLL; I could have also used FFT with peak interpolation.

In my fingerprinting tool, the recording is first split into single transmissions. The CTCSS tone is bandpass filtered and a PLL starts tracking it. When the PLL frequency stops fluctuating, i.e. the standard deviation is small enough, it's

considered locked and its frequency is averaged over this time. The average RMS power is measured similarly.

Here's one such transmission:

At least three clusters are clearly distinguishable by eye. Zooming in to one of the clusters reveals it's made up of several smaller clusters. Perhaps the larger clusters correspond to three different models of radios in use, and these smaller ones are the individual transmitters?

A heat map reveals even more structure:

It seems at least 12 clusters, i.e. potential individual transmitters, can be distinguished.

Even though most transmissions are part of some cluster, there are many outliers as well. These appear to correspond to a very noisy or very short transmission. (Could the FFT have produced better results with these?)

Use as transcription aid

My goal was to make these fingerprints useful as labels aiding transcription. This way, a human operator could easily distinguish parties of a conversation and add names or call signs accordingly.

I experimented with automated k-means clustering, but that didn't immediately produce appealing results. Then I manually assigned 12 anchor points at apparent cluster centers and had a script calculate the nearest anchor point for all transmissions. Prior to distance calculations the axes were scaled so that the data seemed uniformly distributed around these points.

This automatic labeling proved quite sensitive to errors. It could be useful when listing possible transmitters for an unknown transmission with no context; distances to previous transmissions positively mentioning call signs could be used. Instead I ended up printing the raw coordinates and colouring them with a continuous RGB scale:

Here the colours make it obvious which party is talking. Call signs written in a darker shade are deduced from the context. One sentence, most probably by "Cobra 1", gets lost in noise and the RMS power measurement becomes inaccurate (463e-6). The PLL frequency is still consistent with the conversation flow, though.

If CTCSS is not absolutely required in your network, i.e. there are no unwanted conversations on the frequency, then it can be disabled to prevent this type of fingerprinting. In Motorola radios this is done by setting the CTCSS code to 0. (In the menus it may also be called a PT code or Interference Eliminator code.) In many other consumer radios it's doesn't seem to be that easy.

CTCSS is a suitable signal for fingerprinting transmitters, reflecting minute differences in crystal frequencies and, possibly, FM modulation indices. Even a cheap receiver can recover these differences. It can be used when the signal is

The average RMS power is measured similarly.

Oona Räisänen

What makes a great photo editing intern (Apply now for Fall 2016!)

NPR Apps Blog

This is not your standard photo internship!

This internship is an opportunity to learn more about the world of photo editing. Our goal isn't to make you into a photo editor; we view this internship as a chance for you to understand what it is like to be an editor and improve your visual literacy, which can help you become a better photographer.

What you will be doing

Editing: You'll be working closely with the Visuals Team's photo editors (Ariel and Emily) on fast-paced deadlines – we're talking anywhere from 15 minutes to publication, to short-term projects that are a week out. You'll dig into news coverage and photo research, learning how to communicate about what makes a good image across a range of news topics, including international, national, technology, arts and more.

Photography: Depending on the news cycle, there may be opportunities to photograph DC-area assignments. This can mean you'd have one or two shoots in a week, or maybe just a couple shoots in a month. You'll work closely with a radio or web reporter while out in the field, and a photo editor will go through your work and provide feedback for each assignment. There will also be a chance to work on portraiture and still lifes in our studio.

We also encourage each intern to create a self-directed project to work on throughout the semester. It can be an Instagram series, video, photo essay, text story or anything in-between. You can work independently or with another intern or reporter.

You will be part of NPR's intern program, which includes 40-50 interns each semester, across different departments. There will be coordinated training and intern-focused programming throughout the semester, which includes meeting NPR radio hosts, career development and other opportunities. As an intern, you will be treated as a member of the team. Many NPR employees are former interns and they're always willing to help current interns.

Any student (undergraduate or graduate), or person who has graduated no more than 12 months prior to the start of the internship period to which he/she is applying is eligible. Interns must be authorized to work in the United States.

Who should apply

We're looking for candidates that have a strong photojournalism background. An interest in editing, or experience with video/photo editing is a nice plus. It's also helpful if you've completed at least one photojournalism-focused internship prior to applying (let us know if you have!), though it's not necessary. A portfolio, however, is required.

We also want folks who can tell us what they would like to accomplish during their time at NPR. What do you want to learn? What do you want to try? We try to shape each internship around our intern, so we rely on you to tell us what goals you have for your time with us!

So how do I apply?

Does this sound like you? Read our post about how to write a cover letter and then apply now!

The deadline for applications HAS BEEN EXTENDED to May 29, 2016, 11:59pm EST.

Measuring the impact of the .NET Garbage Collector

Matt Warren (.NET)

There is an update to this post, based on feedback I received.

In my last post I talked about the techniques that the Roslyn team used to minimise the effect of the Garbage Collector (GC). Firstly I guess its worth discussing what the actual issue is.

GC Pauses and Latency

In early versions of the .NET CLR, garbage collection was a “Stop the world” event, i.e. before a GC could happen all the threads in your program had to be brought to a safe place and suspended. If your ASP.NET MVC app was in the middle of serving a request, it would not complete until after the GC finished and the latency for that user would be much higher than normal. This is exactly the issue that Stackoverflow ran into a few years ago, in their battles with the .NET Garbage Collector. If you look at the image below (from that blog post), you can see the spikes in response times of over 1 second, caused by Gen 2 collections.

However in the .NET framework 4.5 there were enhancements to the GC brought in that can help mitigate these (emphasis mine)

The new background server GC in the .NET Framework 4.5 offloads much of the GC work associated with a full blocking collection to dedicated background GC threads that can run concurrently with user code, resulting in much shorter (less noticeable) pauses. One customer reported a 70% decrease in GC pause times.

But as you can see from the quote, this doesn't get rid of pauses completely, it just minimises them. Even the SustainedLowLatency mode isn't enough, “The collector tries to perform only generation 0, generation 1, and concurrent generation 2 collections. Full blocking collections may still occur if the system is under memory pressure.” If you want a full understanding of the different modes, you can see some nice diagrams on this MSDN page.

I'm not in any way being critical or dismissive of these improvements. GC is a really hard engineering task, you need to detect and clean-up the unused memory of a program, whilst it's running, ensuring that you don't affect it's correctness in any way and making sure you add as little overhead as possible. Take a look at this video for some idea of what's involved. The .NET GC is a complex and impressive piece of engineering, but there are still some scenarios where it can introduce pauses to your program.

Aside: In the Java world there is a commercial Pauseless Garbage Collector available from Azul Systems. It uses a patented technique to offer “Predictable, consistent garbage collection (GC) behavior” and “Predictable, consistent application response times”, but there doesn't seem to be anything like that in the .NET space.

Detecting GC Pauses

But how do you detect GC pauses, well the first thing to do is take a look at the properties of the process using the excellent Process Explorer tool from Sysinternals (imagine Task Manager on steroids). It will give you a summary like the one below, the number of Gen 0/1/2 Collections and % Time in GC are the most interesting values to look at.

But the limitation of this is that it has no context, what % of time in GC is too high, how many Gen 2 collections are too many? What effect does GC actually have on your program, in terms of pauses that a customer will experience?

jHiccup and HdrHistogram

To gain a better understanding, I've used some of the ideas from the excellent jHiccup Java tool. Very simply, it starts a new thread in which the following code runs:

```
var timer = new Stopwatch (); while ( true ) { timer . Restart (); Thread . Sleep ( 1 ); timer . Stop (); /\ allow a little bit of leeway if ( timer . ElapsedMilliseconds > 2 ) { /\ Record the pause _histogram . recordValue ( timer . ElapsedMilliseconds ); }
```

Any pauses that this thread experiences will also be seen by the other threads running in the program and whilst these pauses aren't guaranteed to be caused by the GC, it's the most likely culprit.

Note: this uses the .NET port of the Java HdrHistogram, a full explanation of what HdrHistogram offers and how it works is available in the Readme. But the summary is that it offers a non-intrusive way of collecting samples in a histogram, so that you can then produce a graph of the 50%/99%/99.9%/99.99% percentiles. It does this by allocating all the memory it needs up front, so after start-up it performs no allocations during usage. The benefit of recording full percentile information like this is that you get a much fuller view of any outlying values, compared to just recording a simple average.

To trigger garbage collection, the test program also runs several threads, each executing the code below. In a loop, each thread creates a large string and a byte array, to simulate what a web server might be doing when generating a response to a web request (for instance from de-serialising some Json and creating a HTML page). Then to ensure that the objects are kept around long enough, they are both put into a Least Recently Used (LRU) cache, that holds the 2000 most recent items.

```
processingThreads [ i ] = new Thread ( () => { var threadCounter = 0 ; while ( true ) { var text = new string ( ( char ) random . Next ( start , end + 1 ) , 1000 ); stringCache . Set ( text . GetHashCode () , text );
```

```
/\ Use 80K, If we are > 85,000 bytes = LOH and we don't want these there var bytes = new byte [ 80 * 1024 ]; random . NextBytes ( bytes ); bytesCache . Set ( bytes . GetHashCode () , bytes );
```

Fuzzing the .NET JIT Compiler

Matt Warren (.NET)

I recently came across the excellent 'Fuzzlyn' project , created as part of the 'Language-Based Security' course at Aarhus University . As per the project description Fuzzlyn is a:

... fuzzer which utilizes Roslyn to generate random C# programs

And what is a 'fuzzer', from the Wikipedia page for 'fuzzing':

Fuzzing or fuzz testing is an automated software testing technique that involves providing invalid, unexpected, or random data as inputs to a computer program.

Or in other words, a fuzzer is a program that tries to create source code that finds bugs in a compiler .

Massive kudos to the developers behind Fuzzlyn, Jakob Botsch Nielsen (who helped answer my questions when writing this post), Chris Schmidt and Jonas Larsen , it's an impressive project!! (to be clear, I have no link with the project and can't take any of the credit for it)

Compilation in . NET

But before we dive into 'Fuzzlyn' and what it does, we're going to take a quick look at 'compilation' in the . NET Framework . When you write C#/VB. NET/F# code (delete as appropriate) and compile it, the compiler converts it into Intermediate Language (IL) code. The IL is then stored in a .exe or .dll, which the Common Language Runtime (CLR) reads and executes when your program is actually run. However it's the job of the Just-in-Time (JIT) Compiler to convert the IL code into machine code.

Why is this relevant? Because Fuzzlyn works by comparing the output of a Debug and a Release version of a program and if they are different, there's a bug! But it turns out that very few optimisations are actually done by the 'Roslyn' compiler , compared to what the JIT does, from Eric Lippert's excellent post What does the optimize switch do? (2009)

The /optimize flag does not change a huge amount of our emitting and generation logic . We try to always generate straightforward, verifiable code and then rely upon the jitter to do the heavy lifting of optimizations when it generates the real machine code. But we will do some simple optimizations with that flag set. For example, with the flag set:

He then goes on to list the 15 things that the C# Compiler will optimise, before finishing with this:

That's pretty much it. These are very straightforward optimizations; there's no inlining of IL, no loop unrolling, no interprocedural analysis whatsoever. We let the jitter team

worry about optimizing the heck out of the code when it is actually spit into machine code; that's the place where you can get real wins .

So in . NET, very few of the techniques that an 'Optimising Compiler' uses are done at compile-time . They are almost all done at run-time by the JIT Compiler (leaving aside AOT scenarios for the time being).

For reference, most of the differences in IL are there to make the code easier to debug, for instance given this C# code:

```
public void M () { foreach ( var item in new [] { 1 , 2 , 3 , 4 })  
{ Console . WriteLine ( item ); } }
```

The differences in IL are shown below ('Release' on the left, 'Debug' on the right). As you can see there are a few extra nop instructions to allow the debugger to 'step-through' more locations in the code, plus an extra local variable, which makes it easier/possible to see the value when debugging.

(click for larger image or you can view the 'Release' version and the 'Debug' version on the excellent SharpLab)

For more information on the differences in Release/Debug code-gen see the 'Release (optimized)' section in this doc on CodeGen Differences . Also, because Roslyn is open-source we can see how this is handled in the code:

All usages of the 'OptimizationLevel' enum

All usage of the 'ILEmitStyle' enum

In Debug builds, extra 'sequence points' are created (as shown above)

Extra field added to the the async/await 'State Machine' in Debug builds

In Release builds, some 'catch' blocks are discarded

In Debug builds, hoisted variables aren't re-used

Extra Attribute is inserted in Debug builds

This all means that the 'Fuzzlyn' project has actually been finding bugs in the . NET JIT, not in the Roslyn Compiler

(well, except this one Finally block belonging to unexecuted try runs anyway , which was fixed here)

How it works

At the simplest level, Fuzzlyn works by compiling and running a piece of randomly generated code in 'Debug' and 'Release' versions and comparing the output. If the 2 versions produce different results, then it's a bug, specifically a bug in the optimisations that the JIT compiler has attempted. If] = 0.17 , [(int) StatementKind . Block] = 0.1 , IL(int \ StatementKind . Call] = 0.1 , IL(int \ StatementKind

GC Pauses and Safe Points

Matt Warren (.NET)

GC pauses are a popular topic, if you do a google search , you'll see lots of articles explaining how to measure and more importantly how to reduce them. This issue is that in most runtimes that have a GC, allocating objects is a quick operation, but at some point in time the GC will need to clean up all the garbage and to do this it has to pause the entire runtime (except if you happen to be using Azul's pauseless GC for Java).

The GC needs to pause the entire runtime so that it can move around objects as part of its compaction phase. If these objects were being referenced by code that was simultaneously executing then all sorts of bad things would happen. So the GC can only make these changes when it knows that no other code is running, hence the need to pause the entire runtime.

GC Flow

In a previous post I demonstrated how you can use ETW Events to visualise what the .NET Garbage Collector (GC) is doing. That post included the following GC flow for a Foreground/Blocking Collection (info taken from the excellent blog post by Maoni Stephens the main developer on the .NET GC):

GC_suspendEE_V1

GC_suspendEEEnd_V1 GC suspension in Runtime code

Inside code that it controls, the runtime inserts method calls to ensure that threads can regularly poll to determine when they need to suspend. For instance take a look at the following code snippet from the `IndexOfCharArray()` method (which is called internally by `String.IndexOfAny(..)`). Notice that it contains multiple calls to the macro `FC_GC_POLL_RET()` :

```
FCIMPL4 ( INT32 , COMString :: IndexOfCharArray ,
StringObject * thisRef , CHARArray * valueRef , INT32
startIndex , INT32 count ) { //
```

```
  /\ use probabilistic map, see (code: InitializeProbabilisticMap)
  int charMap [ PROBABILISTICMAP_SIZE ] = { 0 } ;
```

```
  InitializeProbabilisticMap ( charMap , valueChars , value-
  Length );
```

```
  for ( int i = startIndex ; i = 0 ) { FC_GC_POLL_RET (); return
  i ; } }
```

```
  FC_GC_POLL_RET (); return - 1 ; }
```

There are lots of other places in the runtime where these calls are inserted, to ensure that a GC suspension can happen as soon as possible. However having these calls spread throughout the code has an overhead, so the runtime uses a special trick to ensure the cost is only paid when a suspen-

sion has actually been requested, From `jithelp.asm` you can see that the method call is re-written to a `nop` routine when not needed and only calls the actual `JIT_PollGC()` function when absolutely required:

```
; Normally (when we're not trying to suspend for GC),
the ; CORINFO_HELP_POLL_GC helper points to this nop
routine. When we're ; ready to suspend for GC, we
whack the Jit Helper table entry to point ; to the real
helper. When we're done with GC we whack it back.
PUB- LIC @JIT_PollGC_Nop@0 @JIT_PollGC_Nop@0 PROC ret
@JIT_PollGC_Nop@0 ENDP
```

However calls to `FC_GC_POLL` need to be carefully inserted in the correct locations, too few and the EE won't be able to suspend quickly enough and this will cause excessive GC pauses, as this comment from one of the .NET JIT devs confirms:

GC suspension in User code

Alternatively, in code that the runtime doesn't control things are a bit different. Here the JIT analyses the code and classifies it as either:

Partially interruptible

Fully interruptible

Partially interruptible code can only be suspended at explicit GC poll locations (i.e. `FC_GC_POLL` calls) or when it calls into other methods. On the other hand fully interruptible code can be interrupted or suspended at any time, as every line within the method is considered a GC safe-point.

I'm not going to talk about how the thread-hijacking mechanism works (used with fully interruptible code), as it's a complex topic, but as always there's an in-depth section in the BOTR that gives all the gory details. If you don't want to read the whole thing, in summary it suspends the underlying native thread, via the Win32 `SuspendThread` API .

You can see some of the heuristics that the JIT uses to decide whether code is fully or partially interruptible as it seeks to find the best trade-off between code quality/size and GC suspension latency. But as a concrete example, if we take the following code that accumulates a counter in a tight loop:

```
public static long TestMethod () { long counter = 0 ; for ( int
i = 0 ; i
```

```
public static long TestMethod () { long counter = 0 ; for ( int
i = 0 ; i Further Reading
```

Modern Garbage Collection in Theory and Practice

GC-safe points, mutator suspension and barriers

How local variable usage information is maintained in .net-clr source code

Top 10 AI Tools in 2023 That Will Make Your Life Easier

Unknown · TechCrunch

In this article, we explore the top 10 AI tools that are driving innovation and efficiency in various industries. These tools are designed to automate repetitive tasks, improve workflow, and increase productivity. The tools included in our list are some of the most advanced and widely used in the market, and are suitable for a variety of applications. Some of the tools focus on natural language processing, such as ChatGPT and Grammarly, while others focus on image and video generation, such as DALL-E and Lumen5. Other tools such as OpenAI Codex, Tabnine, Canva, Jasper AI, and Surfer SEO are designed to help with specific tasks such as code understanding content writing and website optimization. This list is a great starting point for anyone looking to explore the possibilities of AI and how it can be applied to their business or project. So let's dive into

1. ChatGPT

ChatGPT is a large language model that generates human-like responses to a variety of prompts. It can be used for tasks such as language translation, question answering, and text completion. It can handle a wide range of topics and styles of writing, and generates coherent and fluent text, but should be used with care as it may generate text that is biased, offensive, or factually incorrect.

Pros:

Generates human-like responses to a variety of prompts

Can be fine-tuned for specific tasks such as language translation, question answering, and text completion

Can handle a wide range of topics and styles of writing

Can generate coherent and fluent text, even when completing a given text prompt.

Cons:

May generate text that is biased or offensive

Can generate text that is not accurate or factually correct

May require large amounts of computational resources to run

The model can sometimes generate text that is not coherent or fluent, depending on the prompt given.

Overall, ChatGPT is a powerful tool for natural language processing, but it should be used with care and with an understanding of its limitations.

2. DALL-E

DALL-E is a generative model developed by OpenAI that is capable of generating images from text prompts. It is based on the GPT-3 architecture, which is a transformer-based neural network language model that has been trained on a massive dataset of text. DALL-E can generate images that are similar to a training dataset and it can generate high-resolution images that are suitable for commercial use.

Pros:

Generates high-resolution images

Can generate images from text prompts

It can be fine-tuned for specific tasks such as generating images of a certain style or category

Cons:

May generate images that are not entirely original and could be influenced by the training data

May require significant computational resources to run

The quality of the generated images may vary depending on the specific prompt

Overall, DALL-E is a powerful AI-based tool for generating images, it can be used for a variety of applications such as creating images for commercial use, gaming, and other creative projects. It is important to note that the generated images should be reviewed and used with care, as they may not be entirely original and could be influenced by the training data.

3. Lumen5

Lumen5 is a content creation platform that uses AI to help users create videos, social media posts, and other types of content. It has several features that make it useful for content creation and marketing, including:

Pros:

Automatically summarizes text from a blog post, article, or other source into a script for a video or social media post

Virtual music box

Oona Räisänen · Oona Raisenen (windytan)

A little music project I was writing required a melody be played on a music box. However, the paper-programmable music box I had (pictured) could only play notes on the C major scale. I couldn't easily find a realistic-sounding synthesizer version either. They all seemed to be missing something. Maybe they were too perfectly tuned? I wasn't sure.

Perhaps, if I digitized the sound myself, I could build a flexible virtual instrument to generate just the perfect sample for the piece!

I haven't really made a sampled instrument before, short of perhaps using Impulse Tracker clones with terrible single-sample ones. So I proceeded in an improvised manner. Below I'll post some interesting findings and sound samples of how the instrument developed along the way. There won't be any source code as for now.

By the way, there is a great explanatory video by engineer-guy about the workings of music boxes that will explain some terminology ("pins" and "teeth") used in this post.

Recording samples

The first step was, obviously, to record the sound to be used as samples. I damped my room using towels and mattresses to minimize room echo; this could be added later if desired, but for now it would only make it harder to cleanly splice the audio. The microphone used was the Audio Technica AT2020, and I digitized it using the Behringer Xenyx 302 USB mixer.

I perforated a paper roll to play all the possible notes in succession, and rolled the paper through. The sound of the paper going through the mechanism posed a problem at first, but I soon learned to stop the paper at just the right moment to make way for the sound of the tooth.

Now I had pretty decent recordings of the whole two-octave range. I used Audacity to extract the notes from the recording, and named the files according to the actual playing MIDI pitch. (The music box actually plays a G# major scale, contrary to what's marked on the blank paper rolls.)

The missing notes

Next, we'll need to generate the missing notes that don't belong in the scale of this music box. Because pitch is proportional to the speed of vibration, this could be done by simply speeding up or slowing down an adjacent note by just the right factor. In equal temperament tuning, this factor would be the 12th root of 2, or roughly 1.05946. Such scaling is straightforward to do on the command line using SoX, for instance (`sox c1.wav c_sharp1.wav speed 1.05946`).

This method can also be used to generate whole new octaves; for example, a transposition of +8 semitones would

have a ratio of $(12 \sqrt[12]{2})^8 \approx 1.5874$. Inter-note variance could be retained by using a random source file for each resampled note. But large-interval transpositions would probably not sound very good due to coloring in the harmonic series.

Here's a table of some intervals and the corresponding speed ratios in equal temperament:

$$\begin{aligned} -3 &= (12 \sqrt[12]{2})^{-3} \approx 0.840896 & -2 &= (12 \sqrt[12]{2})^{-2} \approx 0.890899 & -1 &= (12 \sqrt[12]{2})^{-1} \approx 0.943874 \\ +1 &= (12 \sqrt[12]{2})^1 \approx 1.059463 & +2 &= (12 \sqrt[12]{2})^2 \approx 1.122462 & +3 &= (12 \sqrt[12]{2})^3 \approx 1.189207 \end{aligned}$$

First test!

Now I could finally write a script to play my melody!

(HTML5 audio: Music box notes.)

It sounds pretty good already - there's no obvious noise and the samples line up seamlessly even though they were just naively glued together sample by sample. There's a lot of power in the lower harmonics, probably because of the big cardboard box I used, but this can easily be changed by EQ if we want to give the impression of a cute little music box.

Adding errors

The above sound still sounded quite artificial, I think mostly because simultaneous notes start on the same exact millisecond. There seems to be a small timing variance in music boxes that is an important contributor to their overall delicate sound. In the below sample I added a timing error from a normal distribution with a standard deviation of 11 milliseconds. It sounds a lot better already!

(HTML5 audio: Music box notes.)

Other sounds from the teeth

If you listen to recordings of music boxes you can occasionally hear a high-pitched screech as well. It sounds a bit like stopping a tuning fork or guitar string with a metal object. That's why I thought it must be the sound of the pin stopping a vibrating tooth just before playing another note on the same tooth.

Sure enough, this sound could always be heard by playing the same note twice in quick succession. I recorded this sound for each tooth and added it to my sound generator. The sound will be generated only if the previous note sample is still playing, and its volume will be scaled in proportion to the tooth's envelope amplitude at that moment. Also, it will silence the note. The amount of silence between the screech and the next note will depend on a tempo setting.

Adding this resonance definitely brings about a more organic feel:

(HTML5 audio: Music box notes.)

Complex But Not Dynamic: Using A Static Site To Crowdsource Playgrounds

NPR Apps Blog

This post is cross-posted with our friends at Source .

We usually build relatively simple sites with our app template . Our accessible playgrounds project needed to be more complex. We needed to deal with moderated, user-generated data. But we didn't have to run a server in order to make this site work; we just modified our app template .

App template-based sites are HTML files rendered from templates and deployed to Amazon's Simple Storage Service (S3). This technique works tremendously for sites that never change, but our playgrounds site needs to be dynamic.

When someone adds, edits or deletes a playground, we POST to a tiny server running a Flask application. This application appends the update to a file on our server, one line for each change. These updates accumulate throughout the day.

At 5 a.m., a cron job runs that copies and then deletes this file, and then processes updates from the copied file. (This copy-delete-read the copy flow helps us solve race conditions where new updates from the web might attempt to write to a locked-for-reading file. After the initial copy-and-delete step, any new writes will be written to a new updates file that will get processed the next day.)

Each update is processed twice. First, we write the old and new states of the playground to a revision log with a time-stamp, like so:

```
{ 'slug': 'ambucs-hub-city-playground-at-maxey-park-lub-bock-tx', 'revisions': [ { 'field': 'address', 'from': '26th Street and Nashville Avenue' 'to': '4007 26th Street' }, ], 'type': 'update' }
```

Second, we update the playground in a SQLite database. When this is complete, a script on the server regenerates the site from the data in the database. Since each page includes a list of other nearby playgrounds, we need to regenerate every playground page. This process takes 10 or 15 minutes, but it's asynchronous from the rest of the application, so we don't mind. We're guaranteed to have the correct version of each playground page generated every 24 hours.

At each step of the process, we take snapshots of the state of our data. Before running our update process, we time-stamp and copy the JSON file of updates from the previous day. We also time-stamp and copy the SQLite database file and push it up to S3 for safekeeping.

Maintaining a crowdsourced web site requires a little work. We fix spelling and location errors, remove duplicates, and delete playgrounds that were added but aren't accessible.

Typically, you'd run an admin site for your maintenance tasks, but we decided that our editors use the public-facing

site just like our readers. That said, our editors still need a way to check the updates our users are making.

Since we only process updates once every 24 hours, we decided to just send an email. For additions, we link the playground URL in the email so that editors could click through. For updates, we list the changes. And for delete requests, we include a link that, when clicked, confirms a deletion and instructs the site to process the delete during the next day's cron.

Flat files are awesome, but without a web server, how do you search?

To solve this, we use Amazon's CloudSearch. Eventually, we'll probably implement a way to find playgrounds with certain features or to search by name. But right now, we're using it just for geographic search, e.g., finding playgrounds near a point.

To implement geographic search in CloudSearch you need to use rank expressions , bits of JavaScript that apply an order to the results. CloudSearch allows you to specify a rank expression as a parameter to the search URL. That's right: Our search URLs include a string that contains instructions for CloudSearch to order the results. Amazon has documentation on how to use this to implement simple "great circle" math. We took it a step further and implemented spherical law of cosines because it is a more accurate algorithm for determining distance between points on a sphere.

You can see the source code where we build our search querystrings in the playgrounds repository , but you should take note of a few further caveats.

CloudSearch only supports unsigned integers , so we have to add the 180 degrees (because latitudes and longitudes can be negative numbers) and also multiply the coordinates by 10,000 (because an unsigned integer can't have a decimal point) to get five decimal points of precision. Finally, we have to reverse this process within our rank expression before converting the coordinates to radians to calculate distance.

Also, a single CloudSearch instance is not very stable when running high-CPU queries like geographic searches. During load testing we saw a large number of HTTP 507 errors, indicating that the search service was overloaded . Unfortunately, 5xx errors and JSONP don't mix . To solve this, we catch 507 errors in Nginx and instead return a HTTP 202 with a custom JSON error document. The 202 response allowed us to read the JSON in the response and then retry the search if it failed. We retry up to three times, though in practice we observed that almost every failed request would return a proper result after only a single fail/retry.

Finally, while Amazon would auto-scale our CloudSearch instances to match demand, we couldn't find any published material explaining how often Amazon would spin up new servers or how many would initialize at once. So, we reached out to Amazon. They were able to set our Cloud-

Recurrent Networks Hello World in Clojure with new Deep Diamond RNN support on CPU and GPU

Dragan Djuric

I've been busy in the last period working on new major features in Deep Diamond, one of which is the support for Recurrent Neural Networks (RNN). It's not been an easy ride, but I can finally show you some results! Big thanks for everyone that's helping me with this by buying my books (or subscribing to the upcoming editions), and Clojurists Together, who generously funded me in the past year to work on this.

I know that most of you probably don't have much more than passing familiarity with deep learning, let alone recurrent neural networks, and that's why I'll try to show a very simple example on the level of Hello World that anyone interested in machine learning and programming can understand and try.

So, enough talk, let's get down to business.

What are we doing

We are demonstrating a hammer: recurrent neural networks. Just kidding; we would like to create a (software) device that can learn to predict the next data point in a series. Depending on the data, this can be done in a number of ways (even by convolutional neural networks (CNN) that Deep Diamond already supported), one of which is RNN. So, we are creating a recurrent network, and training it with a set of data for this task.

An example of data that fits this task would be temperature at some place, stock prices, and any other (possibly infinite) sequence of numbers in one of more dimensions that have an ordinal relation, that is, has an abstract notion of time attached to it. Then, we are trying to forecast one or more values that are happening in the future (temperature the next day, or the closing price of a stock, etc.).

Since Hello World has to be dead simple, a real example would have too many opaque numbers, so we'll solve an artificially trivial task of teaching our network to predict the next number in the series for obvious series such as 1, 2, 3, 4, 5. Of course, in real life we rarely need to solve that exact task with such a bazooka as RNN, but if this is your first contact with time series prediction with deep learning, I guess it'd be just what works best.

Let's get down to code

First, we'll require a bunch of Clojure namespaces that we need.

```
(require ' [ uncompile.commons.core :refer [ with-release let-release release ] ] ' [ uncompile.neanderthal [ core :refer [ ge dim amax submatrix subvector mrows trans transfer transfer! view-vctr native view-ge cols mv! rk! raw col row nrm2 scal! ncols dim rows axpby! ] ] [ native :refer [ fge native-float fv iv ] ] ] ' [ uncompile.diamond
```

```
[ tensor :refer [ *diamond-factory* tensor offset! connector transformer desc revert shape input output view-tz batcher ] ] [ dnn :refer [ rnn infer sum activation inner-product dense network init! train cost train train-shuffle ending ] ] [ native :refer [ ] ] )
```

All the AI in the world would be useless to us if we hadn't measured some data from the real world to feed it. Suppose (for the sake of Hello World), that I "measured" a narrow range of whole number domain by generating it from thin air. Does this data tell us anything about stock market? Of course not, and please does not expect any model that we train on this data to magically be informed on anything that could not be learned from the data it has seen.

```
(def simple-sequence ( range -100 100 ) )
```

```
(-100 -99 -98 -97 -96 -95 -94 -93 -92 -91 -90 -89 -88 -87 -86 -85 ...)
```

Now, I'll create a blueprint for an arbitrary recurrent neural network (RNN). This network has 3 recurrent layers (Gated Recurrent Unit cells), an abbreviation to one timestep, and two dense layers at the end. Please note that I used a completely arbitrary architecture. Neither this layer structure, nor the number of hidden neurons are optimal for this data; we are not even sure it's any good. If anything, it's probably a huge overkill. I chose it simply to show you how it's easy to construct with Deep Diamond (), as it will practically do everything on its own if you specify the bare minimum, that is "what you want". And we hope it'll at least learn to work well at the end, as non-optimal as it is.

```
(def net-bp ( network ( desc [ 5 32 1 ] :float :tnc ) [ ( rnn [ 128 ] :gru ) ( rnn 2 ) ( abbreviate ) ( dense [ 128 ] :relu ) ( dense [ 1 ] :linear ) ] ) )
```

There is no place here to explain what RNN is and how it works internally, other than pointing that recurrent layers have an ability to handle sequential relations of its input by "memorizing" the signals that pass through it. The upcoming version of my book Deep Learning for Programmers (2.0) discusses RNNs in more detail.

Formatting the input data

The input of this network differs from fully connected or convolutional networks by explicitly modeling the time dimension, "t" in the ":tnc" format. Technically, you can feed it with any 3D tensor that matches its [5 32 1] shape, but for that data to be in context, it has to actually be arranged as 5 timesteps of a minibatch of 32 samples of 1-dimensional data.

We do have 1-dimensional data, but how do we fit our (range -100 100) sequence to its input? We do have more than 5 timesteps (we have 400), and we are far from 32 samples, since we only have one! We could try to just cram the sequence as-is by doing (transfer! simple-sequence (input net)) but this would be the "garbage in" part of "garbage in

Adding a verb to the dotnet CLI tooling

Matt Warren (.NET)

The dotnet CLI tooling comes with several built-in cmds such as build , run and test , but it turns out it's possible to add your own verb to that list.

Arbitrary cmds

From Intro to . NET Core CLI - Design

The way the dotnet driver finds the command it is instructed to run using dotnet {command} is via a convention; any executable that is placed in the PATH and is named dotnet-{command} will be available to the driver . For example, when you install the CLI toolchain there will be an executable called dotnet-build in your PATH; when you run dotnet build , the driver will run the dotnet-build executable. All of the arguments following the command are passed to the command being invoked. So, in the invocation of dotnet build -native , the -native switch will be passed to dotnet-build executable that will do some action based on it (in this case, produce a single native binary).

This is also the basics of the current extensibility model of the toolchain. Any executable found in the PATH named in this way, that is as dotnet-{command} , will be invoked by the dotnet driver.

Fun fact: This means that it's actually possible to make a dotnet go command! You just need to make a copy of go.exe and rename it to dotnet-go.exe

Yay dotnet go (I know, completely useless, but fun none-the-less)!!

(and yes before you ask, you can also make dotnet dotnet work, but please don't do that!!)

With regards to documentation, there's further information in the 'Adding a Command' section of the Developer Guide. Also the source code of the dotnet test command is a really useful reference and helped me out several times.

Before I go any further I just want to acknowledge the 2 blog posts listed below. They show you how to build a custom command that will compresses all the images in the current directory and how to make it available to the dotnet tooling as a NuGet package:

Using nuget packages in ASP. NET Core

Building a custom dotnet cli tool

However they don't explain how to interact with the current project or access it's output. This is what I wanted to do, so this post will pick up where those posts left off.

Information about the current Project

Any effective dotnet verb needs to know about the project it is running in and helpfully those kind developers at Microsoft have created some useful classes that will parse and examine a project.json file (available in the Microsoft. DotNet. ProjectModel NuGet package). It's pretty simple to work with, just a few lines of code and you're able to access the entire Project model :

```
Project project ; var currentDirectory = Directory . GetCur-
rentDirectory (); if ( ProjectReader . TryGetProject ( current-
Directory , out project )) { if ( project . Files . SourceFiles .
Any () ) { Console . WriteLine ( "Files:" ); foreach ( var file in
project . Files . SourceFiles ) Console . WriteLine ( " {0}" ,
file . Replace ( currentDirectory , "" )); } if ( project . Depend-
encies . Any () ) { Console . WriteLine ( "Dependencies:" );
foreach ( var dependency in project . Dependencies ) { Console .
WriteLine ( " {0} - Line:{1}, Column:{2}" , dependency .
SourceFilePath . Replace ( currentDirectory , "" ), dependen-
cy . SourceLine , dependency . SourceColumn ); } ... }
```

Building a Project

In addition to knowing about the current project, we need to ensure it successfully builds before we can do anything else with it. Fortunately this is also simple thanks to the Microsoft. DotNet. Cli. Utils NuGet package (along with further help from Microsoft. DotNet. ProjectModel which provides the BuildWorkspace):

```
// Create a workspace var workspace = new BuildWork-
space ( ProjectReaderSettings . ReadFromEnvironment ());
```

```
// Fetch the ProjectContexts var projectPath = project .
ProjectFilePath ; var runtimeIdentifiers = RuntimeEnviro-
nmentRidExtensions . GetAllCandidateRuntimeIdentifiers
(); var projectContexts = workspace . GetProjectContextCol-
lection ( projectPath ) . EnsureValid ( projectPath ) .
FrameworkOnlyContexts . Select ( c => workspace . GetRun-
timeContext ( c , runtimeIdentifiers )) . ToList ();
```

```
// Setup the build arguments var projectContextToBuild =
projectContexts . First (); var cmdArgs = new List { project-
Path , "-configuration" , "Release" , "-framework" , project-
ContextToBuild . TargetFramework . ToString ()};
```

```
// Build!! Console . WriteLine ( "Building Project for {0}" ,
projectContextToBuild . RuntimeIdentifier ); var result =
Command . CreateDotNet ( "build" , cmdArgs ). Execute ();
Console . WriteLine ( "Build {0}" , result . ExitCode == 0 ?
"SUCCEEDED" : "FAILED" );
```

When this runs you get the familiar dotnet build output if it successfully builds or any error/diagnostic messages if not.

Integrating with BenchmarkDotNet

Now that we know the project has produced an .exe or .dll, we can finally wire-up BenchmarkDotNet and get it to execute the benchmarks for us:

```
try { Console . WriteLine ( "Running BenchmarkDot-
Net" ); BenchmarkDotNet . Cli . BenchmarkDotNet . Create . T
```

Putting a face on quality

espen · Finn.no Tech

The topic of ensuring quality in the services we provide is a much debated topic in our industry [1]. It tends to focus upon things such as testing and how to best automate it to try and reach a state of zero defects. These initiatives are great and we should all pursue them in our daily work. These debates tend to be very focused upon the technical side and how to prevent the intrusion of defects in our build cycle. A result of the technical focus is that you become abstracted from the real issues at hand and it becomes yet another ideological debate (or even worse yet another pissing contest between believers and non-believers).

At FINN.no we try to always view things from an end-user perspective and technical quality is no different. So what does technical quality mean for our end users and do they really care?

Making dreams come true

A service such as FINN.no is much more than just a site where you view classified ads. It is a service where everyday people make some of their dreams come true. They buy the house they have been dreaming of in order to start a family. It is a place where you buy things to keep your kids safe and comfortable. So in order to answer the previous question, do they care about quality? Naturally. Down time on a Sunday at FINN.no prevents people from buying the house, the car or the boat of their dreams.

You get the picture right? Quality of service means that we deliver on our promise of being a marketplace you can rely on to make some of your dreams come true. Behind the discussions about unit-test coverage there is a family not getting their house, car or boat if you mess things up. When we debate whether to write tests up-front or do waterfall planning there is someone out there not getting their car sold. We get so caught up in technology some times that we actually think that it matters. In order to try and get more in touch with how our quality affects our users we realized we had to do something. When growing from a small company to a larger company you often end up losing track of these things and we needed to correct this.

Continuous improvements a.k.a. Lean

At FINN.no we have some people work solely with teaching us continuous improvement . They try to help every team or department to learn the Lean way of working and resolving issues. By spreading their knowledge across the company they start a lot of cross-team-department-initiatives which would not happen without them.

Bulding quality into your product is one of the essential things towards creating an amazing user experience, but the problem is how do you do this on a regular basis? There are numerous tools available for all kinds of testing and quality analysis of code and things like that. These tools are all good and you should use them. However, they all fail to

bring you the one insight which is the most valueable yo your business: what does your users actually think about what you do.

Our customer service and architectural teams have both been working to try and establish a way of working which ensure continuous improvement in what we do. This has resulted in an initiative to try and visualize how our level of quality effects our users and customers.

FINNback or Tweet Board

One easy way of putting a face on quality was to utilize the Twitter Streaming API which enables us to see what people think about our service. Thanks to the brilliant engineers at Twitter this was easily accomplished in just a few hours and we had a node.js application which displayed tweets about us on monitors/TVs in our cantina and in our reception. Thanks to the awesomeness of the Twitter Streaming API we are able to pull both the profile picture, the profile background image and the number of followers into our application in order to give a more human feel to the tweets coming in. This was critical as it makes the tweets real as they come from real live people who we effect with our work. In the sample screenshot of the FINNback application we get a confirmation of something we are painfully aware of, we should have an app .

Doing this Twitter feed visualization is just a first step and we are working hard towards creating more real-time feed-back visualization in order to make sure every one of our employees knows exactly how our users feel about what we do. We are confident that this is a great way of getting focus on technical quality and to make sure we work even harder to provide a service which makes our users dreams come true.

A Management Maturity Model for Performance

Alex Russell (infrequently noted)

Since 2015 I have been lucky to collaborate with more than a hundred teams building PWA s and consult on some of the world's largest sites. Engineers and managers on these teams universally want to deliver great experiences and have many questions about how to approach common challenges. Thankfully, much of what once needed hand-debugging by browser engineers has become automated and self-serve thanks to those collaborations.

Despite advances in browser tooling , automated evaluation , lab tools , guidance , and runtimes , teams I've worked with consistently struggle to deliver minimally acceptable performance with today's popular frameworks. This is not a technical problem per se — it's a management issue, and one that teams can conquer with the right frame of mind and support.

What is Performance?

Protecting the Commons

Levels of Performance Management Maturity

Level 0: Bliss

Level 1: Fire Fighting

Level 2: Global Baselines & Metrics

Level 3: P75+, Site-specific Baselines & Metrics

Level 4: Variance Control & Regression Prevention

Level 5: Strategic Performance

Uneven Steps, Regression, & False Starts

The Role of Senior Management

Questions for Senior Managers

"o1ly, But Make it Performance"

What is Performance?

It may seem a silly question, but what is performance, exactly?

This is a complex topic, but to borrow from a recent post , web performance expands access to information and services by reducing latency and variance across interactions in a session, with a particular focus on the tail of the distribution (P75+) . Performance isn't a binary and there are no silver bullets.

Only teams that master their systems can make intentional trade-offs. Organisations that serve their tools will tread water no matter how advanced their technology, while groups

that understand and intentionally manage their systems can succeed on any stack. 1

The value of performance is deeply understood within a specific community and in teams that have achieved high maturity. But outside those contexts it can be challenging to communicate. One helpful lens is to view the difference between good and bad performance as a gap between expectations and reality.

For executives that value:

Revenue Performance is a significant revenue contributor . To the extent that a product performs poorly, a fraction of revenue will be lost and the narrower the funnel becomes for all types of conversions. 2

Engagement Poor performance has a well-documented relationship to reduced engagement . The space between good and bad performance is the opportunity for a competitor's product to fill the same opening. An underappreciated aspect of this equation is variance . When a product often performs well but sometimes is slow to respond, users lose trust and use that service less. Consistent performance matters just as much as low average latency.

Design Poor performance is a gap between the responsiveness of Figma mockups and brand reality. Products that perform well are closer to the approved design. Many brands build their reputations on fanatical attention to detail about products and the physical environments they're sold in. A laggy digital experience is out of alignment with these values.

Accessibility Performance is the foundation of access. Users experiencing slow services may be unable to access them in the first place , mooting other work poured into improving ally . Reduced network and device capacity correlate with other access challenges. ally that isn't founded on consistently excellent performance can easily veer into performative, rather than effective, territory.

Performance is rarely the single determinant of product success, but it can be the margin of victory. Improving latency and reducing variance allows teams to test other product hypotheses with less noise. A senior product leader recently framed a big performance win as creating space that allows us to be fallible in other areas.

Protecting the Commons

Like accessibility, security, UI coherence, privacy, and testability, performance is an aggregate result. Any single component of a system can regress latency or create variance, which means that like other cross-cutting product properties, performance must be managed as a commons. The approaches that work over time are horizontal, culturally-based, and require continual investment to sustain.

Teams I've consulted with are too often wrenched between celebration over launching "the big rewrite" and the morn-

Know thy .NET object memory layout (Updated 2014-09-03)

Matt Warren (.NET)

Apologies to Nitsan Wakart, from whom I shamelessly stole the title of this post !

tl;dr

The .NET port of HdrHistogram can control the field layout within a class, using the same technique that the original Java code does.

Recently I've spent some time porting HdrHistogram from Java to .NET, it's been great to learn a bit more about Java and get a better understanding of some low-level code. In case you're not familiar with it, the goals of HdrHistogram are to:

Provide an accurate mechanism for measuring latency at a full-range of percentiles (99.9%, 99.99% etc)

Minimising the overhead needed to perform the measurements, so as to not impact your application

You can find a full explanation of what is done and how point 1) is achieved in the project readme .

Minimising overhead

But it's the 2nd of the points that I'm looking at in this post, by answering the question

How does HdrHistogram minimise its overhead?

But first it makes sense to start with the why, well it turns out it's pretty simple. HdrHistogram is meant for measuring low-latency applications, if it had a large overhead or caused the GC to do extra work, then it would negatively affect the performance of the application it was meant to be measuring.

Also imagine for a minute that HdrHistogram took 1/10,000th of a second (0.1 milliseconds or 100,000 nanoseconds) to record a value. If this was the case you could only hope to accurately record events lasting down to a millisecond (1/1,000th of a second), anything faster would not be possible as the overhead of recording the measurement would take up too much time.

As it is HdrHistogram is much faster than that, so we don't have to worry! From the readme :

Measurements show value recording times as low as 3-6 nanoseconds on modern (circa 2012) Intel CPUs.

So how does it achieve this, well it does a few things:

It doesn't do any memory allocations when storing a value, all allocations are done up front when you create the histogram. Upon creation you have to specify the range of measurements you would like to record and the precision. For instance if you want to record timings covering the

range from 1 nanosecond (ns) to 1 hour (3,600,000,000,000 ns), with 3 decimal places of resolution, you would do the following:

```
Histogram histogram = new Histogram(3600000000000L, 3);
```

Uses a few low-level tricks to ensure that storing a value can be done as fast as possible. For instance putting the value in the right bucket (array location) is a constant lookup (no searching required) and on top of that it makes use of some nifty bit-shifting to ensure it happens as fast as possible.

Implements a slightly strange class-hierarchy to ensure that fields are laid out in the right location. If you look at the source you have AbstractHistogram and then the seemingly redundant class AbstractHistogramBase, why split up the fields up like that? Well the comments give it away a little bit, it's due to false-sharing

False sharing

Update (2014-09-03): As pointed out by Nitsan in the comments, I got the wrong end of the stick with this entire section. It's not about false-sharing at all, it's the opposite, I'll quote him to make sure I get it right this time!

The effort made in HdrHistogram towards controlling field ordering is not about False Sharing but rather towards ensuring certain fields are more likely to be loaded together as they are clumped together, thus avoiding a potential extra read miss.

So what is false sharing, to find out more I recommend reading Martin Thompson's excellent post and this equally good one from Nitsan Wakart. But if you're too lazy to do that, it's summed up by the image below (from Martin's post).

The problem is that a CPU pulls data into its cache in lines, even if your code only wants to read a single variable/field. If 2 threads are reading from 2 fields (X and Y in the image) that are next to each other in memory, the CPU running a thread will invalidate the cache of the other CPU when it pulls in a line of memory. This invalidation costs time and in high-performance situations can slow down your program.

The opposite is also true, you can gain performance by ensuring that fields you know are accessed in succession are located together in memory. This means that once the first field is pulled into the CPU cache, subsequent accesses will be cheaper as the fields will be "Hot". It is this scenario HdrHistogram is trying to achieve, but how do you know that fields in a .NET object are located together in memory?

Analysing the memory layout of a .NET Object

To do this you need to drop down into the debugger and use the excellent SOS or Son-of-Strike extension. This is because the .NET JITter is free to reorder fields as it sees fit, so the order you put the fields in your class does not determine the order they end up. The JITter changes the

Summer Project 2016: Customer Insight

Martin Ek · Finn.no Tech

This year's summer intern team consisted of five technology students from UiO and NTNU and one in-house design intern. We have released continuously throughout an eight week period, and have ended with a final solution which is available to the public.

The problem presented for this year's summer project was; What can FINN offer professional vendors at Torget to motivate them to continue advertising as professional vendors. To help us get started, we had a few meetings with FINN's innovation team to work out an ambition and a few ideas. Our ambition was to create value for professional vendors that use Torget as a market to run their business. To achieve this we aimed to provide better advertisement insight for vendors, and give them a better look into their general performance at Torget.

Our application consists of three main parts. The first presents geographical data about the ad's visitors, shown on a map of Norway. This is primarily shown as percentages on each of Norway's five regions, with more specific information shown in the cat's conversation box. The latter also changes when the user clicks around the map. The second part of the application shows demographic data about the customers, which includes age and gender. Here we hope to open a more insightful relationship between seller and customer, which might for example allow sellers to properly target their correct customer group when advertising new products.

The last part of the application is a competitive ranking list that compares ads in the user's category based on number of visits. This is shown with green text to the right of each ad in the list. The user also has the option to change which category they want to see the ranking for, in the blue menu. One of the main goals here is to give users an incentive towards comparing their advertisements to their competitors', by letting them see what part of the market their customers prefer.

Implementing the application

The architecture we went with for the application can mostly be divided into two parts: One that collects statistical data, and one that presents that data to the customer. These meet in the middle through a PostgreSQL database.

The collecting part of the architecture is made up of a Java application that listens to events using FINN's messaging broker, Kafka. This is usually referred to as a consumer. In our situation the consumer receives a message every time a user clicks on a sales ad, so that we can continue to aggregate anonymous statistical data for that specific ad.

The presenting part is mostly made up of a three layer hierarchy. At the bottom we have a Java REST-API that reads from the database, which is built using Spring Boot. The next layer is a Node.js application, which is responsible for

responding to all customer facing requests. Upon receiving a visit this application performs an initial server-side render of the website, which is built using React. This also includes a primary API request, so as to be able to present a mostly complete website to the user on the first request.

This has the positive side effect that the application works decently even in browsers with JavaScript disabled. Upon receiving the initial render of the website the browser client takes over, which we consider the topmost layer of the presentational hierarchy. From here we fire off more API requests, to retrieve the data which is needed to present the rest of the website. This is an important step in maintaining a low average response time, as the user will be able to use most of the application while waiting for the other API requests to finish.

The geographical map and the statistical charts are made using D3.js. The map itself is written using pure D3, while the charts make use of the React friendly wrapper Victory. The data that lays the foundation for the map is transformed from raw Kartverket data to a D3 friendly format, using TopoJSON, a library made by the same author as D3. This allows us to build interactions with the map using regular D3 code, which we at the moment utilise to merge the Norwegian counties into larger regions, and to react to the user's hover and click events.

The front- and backend applications are deployed in Docker containers, and managed through FINN's new Kubernetes cluster. This follows FINN Infrastructure's new deployment strategy, and has made our meeting with pipelines and deployment work easy.

The designers' perspective

When creating new features, technology is obviously important, but making sure people can use it is just as essential. As the team's designers, keeping the user in mind was our priority number one. We spent the summer sketching, prototyping, testing, and continuously iterating to make the final design as user friendly as possible. This required a close collaboration with the developers in the team and attention to detail.

We started out with an idea workshop where we sketched out different layout suggestions. This worked as the basis for further development of the user interface. We then made clickable prototypes in Adobe Experience Design and Invision, which were used in both formal user tests in FINN's usability lab and more spontaneous ones in the busy streets of Oslo. The purpose of these tests was to assess people's understanding of the information presented to them on the screen, as well as general navigation and interaction with the interface. Based on this feedback we were able to improve the different features in the app and ended up with a result we're very pleased with.

Final thoughts - this has been a fun summer!

Capturing PAL video with an SDR (and a few dead-ends)

Oona Räisänen · Oona Raisanen (windytan)

I play 1980s games, mostly Super Mario Bros., on the Nintendo NES console. It would be great to be able to capture live video from the console for recording speedrun attempts. Now, how to make the 1985 NES and the 2013 MacBook play together, preferably using hardware that I already have? This project diary documents my search for the answer.

Here's a spoiler – it did work:

Things that I tried first

A capture device

Video capture devices, or capture cards, are devices specially made for this purpose. There was only one cheap (30€) capture device for composite video available locally, and I bought it, hopefully. But it wasn't readily recognized as a video device on the Mac, and there seemed to be no Mac drivers available. Having already almost capped my budget for this project I then ordered a 5€ EasyCap device from eBay, as there was some evidence of Mac drivers online. The EasyCap was still making its way to Finland as of this writing, so I continued to pursue other routes.

PS: When the device finally arrived, it sadly seemed that the EasyCapViewer-Fushicai software only supports opening this device in NTSC mode. There's PAL support in later commits in the GitHub repo, but the project is old and can't be compiled anymore as Apple has deprecated QuickTime.

Even when they do work, a downside to many cheap capture devices is that they can only capture at half the true frame rate (that is, at 25 or 30 fps).

CRT TV + DSLR camera

The cathode-ray tube television that I use for gaming could be filmed with a digital camera. This posed interesting problems: The camera must be timed appropriately so that a full scan is captured in every frame, to prevent temporal aliasing (stripes). This is why I used a DSLR camera with a full manual mode (Canon EOS 550D in this case).

For the 50 Hz PAL television screen I used a camera frame rate of 25 fps and an exposure time of 1/50 seconds (set by camera limitations). The camera will miss every other frame of the original 50 fps video, but on the other hand, will get an evenly lit screen every time.

A Moiré pattern will also appear if the camera is focused on the CRT shadow mask. This is due to interference between two regular 2D arrays, the shadow mask in the TV and the CCD array in the camera. I got rid of this by setting the camera on manual focus and defocusing the lense just a bit.

This produced surprisingly good quality video, save for the slight jerkiness caused by the low frame rate (video). This

setup was good for one-off videos; However, I could not use this setup for live streaming, because the camera could only record on the SD card and not connect to the computer directly.

LCD TV + webcam

An old LCD TV that I have has significantly less flicker than the CRT, and I could have live video via the webcam. But the Microsoft LifeCam HD-3000 that I have had only a binary option for manual exposure (pretty much “none” and “lots”). Using the higher setting the video was quite washed out, with lots of motion blur. The lower setting was so fast that it looked like the LCD had visible vertical scanning. Brightness was also heavily dependent on viewing angle, which caused gradients over the image. I had to film at a slightly elevated angle so that the upper part of the image wouldn't go too dark, and this made the video look like a bootleg movie copy.

Composite video

Now to capturing the actual video signal. The NES has two analog video outputs: one is composite video and the other an RF modulator, which has the same composite video signal modulated onto an AM carrier in the VHF television band plus a separate FM audio carrier. This is meant for televisions with no composite video input: the TV sees the NES as an analog TV station and can tune to it.

In composite video, information about brightness, colour, and synchronisation is encoded in the signal's instantaneous voltage. The bandwidth of this signal is at least 5 MHz, or 10 MHz when RF modulated, which would require a 10 MHz IQ sampling rate.

I happen to have an Airspy R2 SDR receiver that can listen to VHF and take 10 million samples per second - could it be possible? I made a cable that can take the signal from the NES RCA connector to the Airspy SMA connector. And sure enough, when the NES RF channel selector is at position “3”, a strong signal indeed appears on VHF television channel 3, at around 55 MHz.

Software choices

There's already an analog TV demodulator for SDRs - it's a plugin for SDR# called TVSharp. But SDR# is a Windows program and TVSharp doesn't seem to support colour. And it seemed like an interesting challenge to write a real-time PAL demodulator myself anyway.

I had been playing with analog video demodulation recently because of my HDMI Tempest project (video). So I had already written a C++ program that interprets a 10 Msps digitised signal as greyscale values and sync pulses and show it live on the screen. Perhaps this could be used as a basis to build on. (It was not published, but apparently there is a similar project written in Java, called TempestSDR)

Leaving the Tower of Babel

Morten Lied Johansen · Finn.no Tech

Here at FINN we use mostly Java for our day-to-day work, but we do have some applications and modules written in other languages. We currently have code in Java, Ruby, JavaScript, Objective-C and Scala, supporting things as diverse as testing frameworks and iOS-apps.

Recently there has been some discussion with regards to what we should be using for new projects, as each team of developers start suggesting that they should use something they think would be easier and more effective. The arguments against are based on how easy it would be to recruit new developers who know these new technologies, how well it would perform under our kinds of loads, and whether the assumption that we can be more effective is actually true.

Our strategy is firm on this point, that experiments can be done, but for major applications serving the public, we should be using Java for the time being. We are however considering if it's time to take a second look at our chosen languages, and see if we should expand our portfolio. This strategy is founded on a wish to reduce the size of our technology portfolio, so that we don't have to spend time and effort to bring developers up to speed when they switch teams. There's also a higher chance that new hires will be familiar with Java, while other technologies would often require a period of training when they first start.

During these discussions, we created a quick poll and sent it out on our internal discussionboard. The poll asked questions like "How long have you worked as a developer?", "Which language do you use for your day-to-day work?", "If you were free to choose, which language would you use for your next project?" and "Which programming languages do you know?". Our definition of "know" was very open in this poll, lowering the bar to allow people who had maybe written a single example program, and understands a bit of code could check the box.

Out of the nearly 100 people that work with development, 56 answered. We can assume that the people who did answer, were the people who were interested in the topic, and might not be a representative selection, but the results are interesting none the less.

So.. what did we learn?

Being a poll made in the midst of a discussion, mostly for fun, this part of the poll wasn't framed in a way that gives us much useful information. We can say that the average developer who answered has worked for around seven or eight years, but with what looks like a reasonably fair spread across all groups. Almost as many with a couple years experience, as there are people with 10 to 15 years.

Primary language in day-to-day work

Being a predominately Java shop, most of the respondents were using Java for their daily work. 35 out of 56 were Java-developers. 11 respondents worked with JavaScript daily, while four worked with Scala, three with Ruby, two with Objective-C, and finally one database-developer who worked mostly in T-SQL (The SQL-variant used in Sybase).

Most popular choice if allowed to choose freely

The most interesting part of the poll, with many interesting insights. This is also the most loaded part, as the results could easily short-circuit any discussion about future choice in language.

The bad news (or good, depending on your point of view), is that there was no clear answer from this section. Keep in mind that around 40 people didn't answer this poll, and could be assumed to be content with the current status-quo.

The biggest group was Java, not surprisingly. The surprising part is that it was only 16 people who would choose Java if they could choose freely. This is much lower than expected, but still make up the largest group. Of these, 13 people are already using Java today. 20 Java developers would like to use something else.

So, if Java was the largest group, at 16 people, what does that mean for the remainder of the results? Well, 10 people would have chosen Scala, while JavaScript, Ruby, Clojure and Groovy clock in at about four or five respondents wanting to use them. At the bottom of the pack we have Python and Objective-C with three and two respondents choosing them.

Six people were interested enough to answer the poll, but chose the non-committal "I don't care, as long as I'm making good stuff for our users" option.

A little over three fourths know JavaScript, while Ruby and T-SQL was known to about half.

Morten Lied Johansen

The number of possible answers here was a rather large selection of popular and not-so-popular-but-quite-well-known languages, so the fact that the list only includes eight languages is a sign that it's not a completely random selection. Still, quite a lot of discussion is needed before any one of those languages gets center stage.

A couple curious details found in this part of the poll includes the fact that the only people who would choose JavaScript are people who are already working in JavaScript every day. Groovy, Clojure, Objective-C and Scala have all managed to be chosen by a person who doesn't actually know the language. There's only three people who would switch to Java, if allowed to choose.

Which languages do we "know"?

Adventures in Benchmarking - Memory Allocations

Matt Warren (.NET)

For a while now I've been involved in the Open Source BenchmarkDotNet library along with Andrey Akinshin the project owner. Our goal has been to produce a .NET Benchmarking library that is:

Easy-to-use

First and foremost we do everything we can to ensure that BenchmarkDotNet gives you accurate measurements, everything else is just "sprinkles on the sundae". That is, without accurate measurements, a benchmarking library is pretty useless, especially one that displays results in nanoseconds.

But once point 1) has been dealt with, 2) it's a bit more subjective. Using BenchmarkDotNet involves little more than adding a [Benchmark] attribute to your method and then running it as per the Step-by-step guide in the GitHub README. I'll let you decide if that is easy-to-use or not, but again it's something we strive for. Once you're done with the "Getting Started" guide, there is also a complete set of Tutorial Benchmarks available, as well as some more real-world examples for you to take a look at.

Being "Helpful"

But this post isn't going to be a general BenchmarkDotNet tutorial, instead I'm going to focus on some of the specific tools that it gives you to diagnose what is going on in a benchmark, or to put it another way, to help you answer the question "Why is Benchmark A slower than Benchmark B?"

String Concat vs StringBuilder

Let's start with a simple benchmark:

```
public class Framework_StringConcatVsStringBuilder
{ [ Params ( 1 , 2 , 3 , 4 , 5 , 10 , 15 , 20 ) ] public int Loops ;
```

```
[ Benchmark ] public string StringConcat () { string result =
string . Empty ; for ( int i = 0 ; i < Loops ; i ++ ) result = result + " " + i ; return result ; }
```

But what about a less well-known example. Imagine after some re-factoring you noticed that your application was triggering a lot more Gen 0/1/2 collections (you do monitor this in your live systems right?) After looking at the recent code commits and carrying out some profiling you narrow the problem down to a refactoring that changed a variable declaration from Dictionary to IDictionary, i.e. exactly the type of refactoring that this Stack Overflow question is discussing.

To benchmark what's actually going on here, we can write some code like so:

```
public class Framework_DictionaryVsIDictionary { Dictionary<string, string> dict ; IDictionary<string, string> idict ;
```

```
[ Setup ] public void Setup () { dict = new Dictionary<string, string> (); idict = (IDictionary<string, string>) dict ; }
```

```
[ Benchmark ] public Dictionary<string, string> DictionaryEnumeration ()
{ foreach ( var item in dict ) { ; } return dict ; }
```

```
[ Benchmark ] public IDictionary<string, string> IDictionaryEnumeration ()
{ foreach ( var item in idict ) { ; } return idict ; }
```

Note: we are deliberately not doing anything with the items inside the foreach loop because we just want to see what the difference in iteration of the 2 collections is. Also note that we are using the same underlying data structure, we are just accessing via an IDictionary cast in the 2nd benchmark.

So what results do we get:

Nice and clear, accessing the same data via the IDictionary interface causes a lot of extra allocations, roughly 22 bytes per foreach loop. This in turn triggers a lot of extra GC collections. It's worth pointing out that when BenchmarkDotNet executes, it will run the same benchmark method, IDictionaryEnumeration() in this case, millions of times, so that we can obtain an accurate measurement. Therefore the actual # of Gen 0 collections isn't so important, it is the relative amount compared to the DictionaryEnumeration() benchmark that should be looked at.

Now this scenario might seem a bit contrived and I have to admit that I knew the answer before I started investigating it, however it did originate from a real-life issue, discovered by Ben Adams. For the full background take a look at the CoreCLR GitHub issue, Avoid enumeration allocation via interface, but as shown below this was identified because in Kestrel/ASP.NET the request/response headers are kept in an IDictionary data structure and so cause an additional 128 MBytes of garbage per second, when running at 1 Million requests per/second.

Finally, what is the technical explanation of the additional allocations, quoting from Stephen Toub of Microsoft

and then further down the same issue

Yes, the issue isn't just enumerator allocations, it's also interface-based dispatch. In addition to boxing the enumerator, the MoveNext and Current calls made per element go from being potentially-inlineable non-virtual calls to being interface calls.

Update Feb 2017 - This section is now out-of-date as the implementation details have now changed, please see Adam Sitnik's blog post for all the details

This is all made possible by the excellent Garbage Collection ETW Events that the .NET runtime produces. In particular the GCAllocationTick_V2 Event that is fired each time approximately 100 KB is allocated. An xml representation of a typical event is shown below, you can see that 0x1A060 or 106,592 bytes have just been allocated.

WTF er Programmatic?

Julie Lundgren og Tale Gjøvik · Finn.no Tech

... og hvorfor trenger jeg å bry meg om det?

For å forstå programmatisk kjøp og salg av annonser, er det viktig å forstå hva vi legger i begrepene. Hva skiller det eksempelvis fra tradisjonelt merkevaresalg? Er det ene alternativet bedre enn det andre?

I korte trekk er det viktig å forstå forskjellene mellom direktesalg og programmatisk salg. Direktesalg omfatter alle salg som gjøres fra et menneske til et annet menneske, og er ofte relasjonsbasert. Toyota ringer selger Y med ønske om bannerannonsering på FINN. Selger Y lager et tilbud basert på kundens behov og målsetninger, får deretter godkjenning fra kunden, booker det inn i ROSE og får til slutt en annen medarbeider på Schibsted sin traffic-avdeling til å sette den avtalte annonsen LIVE i annonsestyringssystemet AppNexus.

FINN sine annonseprodukter kan i all hovedsak deles inn i to hovedretninger, men begge starter med samme utgangspunkt. Annonseren velger et banner, eksempelvis netboard. Dette banneret kan du deretter velge om du vil sette i en kontekstuell plassering på FINN, eksempelvis på et bilmerke, eller om du vil vise budskapet til en gitt målgruppe. Se illustrasjon nedenfor.

Programmatisk kjøp og salg er en form for automatisert kjøp av bannerannonser, hvor det i teorien er minimalt med menneskelig kontakt. Programmatisk er ikke et eget

produkt, men heller en annen måte å kjøpe de overnevnte merkevareproduktene til FINN på.

Det er heller ikke tilgjengelig for alle kundene våre. Hvorfor ikke? Jo, det fordrer at Toyota har tilgang til et teknikk-system som kalles «Demand Side Platform» (DSP), hvor de selv enkelt kan velge ønskede medier og målgrupper uten å involvere en selger. Kunden kan kjøpe disse medieplasseringene – og målgruppene – automatisert fordi mediene benytter seg av en teknisk motpart, nemlig en «Sell Side Platform» (SSP) til å tilgjengeliggjøre varelageret sitt i en såkalt annonsebørs. Når rett bruker dukker opp på rett tid, settes det i gang en «auksjon» hvor de ulike kundene (via sine DSP-er) har en budkrig for å nå drømmebrukeren sin. Den som vinner budkrigen får vise annonsen sin til brukeren. Se illustrasjon nedenfor.

Hvorfor kutter vi ikke ut salgsleddet og selger kun programmatisk?

Det er flere grunner til at programmatisk enn så lenge står for en liten del av totalinntektene til FINN Salg. Først og fremst koster en DSP penger, og det benyttes hovedsakelig av profesjonelle merkevarekjøpere i mediebyrå som håndterer såpass store reklameinvesteringer at de kan forsvare teknikk-kostnadene. En bilforhandler fra Oppdal har neppe råd til disse systemene, og vil nok heller ikke tjene på en slik investering. I tillegg er programmatisk kjøp og salg fremdeles ungt i Norge, det krever opplæring i systemene, og i praksis er det

Representative Line: Greater Than False

Remy Porter · The Daily WTF

Today's anonymous submitter passes us a single line of JavaScript, and it's a doozy. This line works, but that's through no fault of the developer behind it.

Pedantically, this is JSX, not pure JavaScript, but the rules still apply.

So, fun fact in JavaScript: `true > 0` is true, and `false > 0` is false. Which generally makes sense, but why would you use that here? But this code is worse than it looks, thanks to operator precedence.

The highest precedence operation is the optional chain-`arr?.length`. The second highest operation? `>`. So the first part of the comparison that evaluates is `false > 0`. Which is false. Do you know what's next? `===`. So we compare `shouldNotShow` to `false`. Then we `&&` that with the potentially falsy value from our `arr?.length`.

It's all a mess, and it's all so we can compare against `false`, which we could have just done with a `!` operator. `!(arr?.length && shouldNotShow)`.

Our submitter credits this to an offshore team, and this does have the vibe of throwing characters at the problem until it passes the test. Less LLM-guided and more "manually executed Markov chain". That's also an accurate description of the rest of the code in this code base: hand crafted Markov chain generation.

[Advertisement] Plan Your .NET 9 Migration with Confidence Your journey to .NET

9 is more than just one decision. Avoid migration migraines with the advice in this free guide. Download Free Guide Now!

Then we && that with the potentially falsy value from our arr?.

Remy Porter

Åpen fagkveld hjemme hos FINN

Gunn Skinderviken · Finn.no Tech

Tradisjonen tro åpner vi også i år dørene hjemme hos oss i Grensen for å dele våre erfaringer rundt hvordan vi jobber med produktutvikling. Teknologi er i fokus, og vi gleder oss til å vise dere hvordan vi bygger - og holder et av Norges mest besøkte nettsted vedlike. En kveld er dedikert studenter, og en de mer erfarne.

GDPR

Sosial rekruttering

Universell utforming

Dette ser vi etter når vi ansetter utviklere

Åpen fagkveld 28. februar kl 16-20 (for de som jobber med teknologi)

16:00 Velkommen til FINN ved Nicolai Høge (CTO)

16:30 Mat og drikke

17:00 Sosial rekruttering
Hvordan vi tok i bruk våre egne data for å lage FINN Jobbmatch.

17:30 GDPR Samtykke - Hvorfor må vi ha det og hvordan håndterer FINN det i en mikroservices-verden?

17:30 Unleash Verktøy/rammeverk for A/B testing.

18:00 Oppetid Hvordan arbeider vi for høy oppetid i FINN. Ambisjoner, målinger og oppfølging.

18:00 Podium Frontend i en mikroservices-verden. Hvordan FINN bygger en unison frontend på kryss av microservicer og team med forskjellige eierskap.

Åpen studentkveld 7. mars kl 16-20 (for studenter)

Program (endringer kan komme)

16:00 Velkommen til FINN ved Nicolai Høge (CTO)

16:30 Mat og drikke

17:00 Dette ser vi etter når vi ansetter utviklere

17:30 Sosial rekruttering
Hvordan vi tok i bruk våre egne data for å lage FINN Jobbmatch

17:30 Universell utforming - hvorfor og hvordan man skal lage noe som funker for alle. Absolutt ALLE!

18:00 Unleash Verktøy/rammeverk for A/B testing

18:00 Podium Frontend i en mikroservices-verden. Hvordan FINN bygger en unison frontend på kryss av microservicer og team med forskjellige eierskap.

Takk til alle dere som kom.

Karrieresiden vår inneholder mer info om oss.

Finn.no wanted to split up their monolith, you won't believe what happens next

Audun Fauchald Strand · Finn.no Tech

After working several years with (successfully) breaking up the legacy monolith, called iAD, Finn.no wanted to add structure to this process, and make sure that all 20 development teams work towards the same target. The target is a microservices architecture with small-ish autonomous services, owning their own data.

With that in mind we arranged our own two-day architecture summit for all Lead Developers, with a combination of external and internal presenters. The focus was on microservices, domain driven design, event driven architectures and modern programming techniques.

We started off the first day with a long session. Finn.no's Chief Enterprise Architect Sebastian Verheughe wanted to communicate his vision. This included topics such as strategic domain-driven-design, data-highways, and how to use event-driven architecture to further decouple our services.

This theme continued, with the first external presenter. Jan Ove Skogheim from Particular Software talked about his experiences from Rikstoto, and how they had broken up a monolith into multiple small services. He also talked about the composite-UI pattern.

Is RESTful

Owns its own data-store

Team ownership

The first day of the summit was concluded with a presentation of the CQRS architectural pattern. This presentation was held by Idar Borlaug and Andreas Berre from WebStep. They showed all the central concepts in CQRS, such as commands, event-stores, aggregates, sagas using very good examples to illustrate their concepts.

The second day of the summit started off with some hardcore programming. The frameworks of choice was node.js and RxJava. Christopher Kolstad started with an example of how a microservice could be implemented using Reactive Streams and RxJava. After that Phillip Johnsen presented a case using node.js in a real time context. These two talks gave a good introduction to new concepts that are not widely used in finn.no today.

During the second day several lightning talks were held, whenever there was a gap in the program. The talks, included topics as diverse as:

Testing of MicroServices

Unleash - Feature Toggles

Event Driven Architecture in Finn.no

Docker and Microservices.

Two more presentations should be mentioned. Firstly, one of the app-developers at Finn.no Haakon Eriksen, shared his opinions on what a perfect service looks like from a frontend developers viewpoint. The key point here was that the service developers should talk to their clients

From Wordpress to Jekyll

Andersos · Finn.no Tech

Last year we decided to move our developer blog from Wordpress to Jekyll . One of the main reasons were the constant security vulnerabilities of Wordpress. We were hosting Wordpress on our own hardware. We pleased our operations team when they could shut down the service. Github provides hosting for Jekyll projects and that was one of many reasons we landed on that solution.

Other reasons why Jekyll is a good fit:

Static files (blazing fast)

Version controlled files (git) and the source is available here

Github handles username and passwords

Easy to run locally

You can use your favourite editor to write in Markdown or just plain HTML

Github restricts the number of plugins you can use down to this list . We really just wanted a simple blog platform so that was no deal breaker. We even get both @Andersos (@mentions) and :+1: (Emoji).

We landed on using Disqus for comments which is loaded on the client side . We made a design template post . We use that post to check out the design of different elements. That post is also a good starting point for people new to markdown. We are using Html-proofer to test the HTML of the site. Html-proofer checks that all the urls and images on the site are work-

ing. Another tip for improving blogposts is to use a tool like Hemingwayapp .

This developer blog is a place where we at Finn.no can share what we are working on and our interests. If you have suggestions or ideas for what we should write about please leave a comment under this post.

List of 10 most visited posts:

The Ultimate view tiles 3

XSS protection whos responsibility

Dependency injection with constructors

Conquering the kingdom of Java with a Node.js trojan horse

When do you know how much testing is enough

Given the git

Log4j2 in production making it fly

I wish I knew my consumers Maven reverse dependency

Email providers in Norway

Dark launching and feature toggles

Suggest.el: Synthesising Constants

Wilfred Hughes

Suggest.el v0.4 is now out, and it offers some really interesting new ways of making suggestions.

Suppose the user gives us the input '(a b c d)' and desired output 'a'. We would already suggest 'car', but that only gets the first element of the list. They may have wanted 'elt' or 'nth', which get elements at a specific position.

We now try adding constants to the user's inputs, specifically 'nil', 't', '-1', '0', '1' and '2'. This makes suggest.el much more effective.

Here's the example we mentioned:

;; Inputs (one per line): '(a b c d)

;; Desired output: 'a

;; Suggestions: (car '(a b c d)) (elt '(a b c d) 0) ;
Ranking Suggestions

Now we have more possibilities, ordering our suggestions is more complex. The first prototype didn't always get the ordering correct:

;; Inputs (one per line): 0

;; Desired output: 1

;; Suggestions: (+ 0 1) ;
Smarter Search

If suggest.el tries a function and it returns, we save it. If it isn't the value we're looking for, it's saved to a list of 'intermediate' values. This is important for finding nested function calls.

Some intermediate values are very common, espe-

cially when exploring arithmetic. There are many ways you can convert an integer to the same float value, for example:

```
( sqrt 0 ) ;=> 0.0 ( float 0 ) ;=> 0.0 ( fround 0 ) ;=> 0.0 ( ffloor 0 ) ;=> 0.0 ( fceiling 0 ) ;=> 0.0 ( ftruncate 0 ) ;=> 0.0
```

Suggest.el previously considered every single way of generating the same value. v0.4 only tries each unique value 3 times. This allows us to explore more unique possibilities, increasing the likelihood of finding a result before giving up.

Finally, I've spent some time researching similar tools, and documented them in the related projects section .

Not only has it been interesting to see other approaches, suggest.el has benefited from the comparisons. The Smalltalk Finder, for example, also explores bitwise operations, so suggest.el does too!

If there are simple code snippets that you think suggest.el should find, please file a bug . I'm routinely surprised by the results it finds, but I'm sure it could be smarter still.

We are using Html-proofer to test the HTML of the site.

Andersos

We're looking for a developer to help us build Carebot

NPR Apps Blog

We're looking for a programmer to join our team for a few months.

Your mission? Break the news's addiction to pageviews, by bringing meaningful analytics to journalists.

Why?

At NPR Visuals, our goal is to make people care. To get them to give a shit about tough problems and people they've never met. It's our job to create empathy in world.

If that's our goal, how do we know if we're accomplishing it? How do we celebrate success?

Enter the Carebot!

Because what you choose to celebrate is super important. If your organization celebrates pageviews, people will create work that gets more pageviews. But it's not our job to get clicks. Our job is to touch hearts. And so we must celebrate stories that do that.

Basic web analytics don't help us do that. So we applied for a Knight grant to build something we're calling Carebot.

Carebot will be a little system for gathering, analyzing and distributing better analytics. (Specifically, there'll be some javascript, some back-end server and API stuff, and a bunch of notification things like email and Slack bots and stuff.) We've only got a few months to work on it, so we're building a prototype. It will help us test an idea: that better analytics make for better journalism.

Who? When? Where?

Carebot will be built by a small team next winter/spring. You'll be working closely with UX expert Livia Labate, our lead architect David Eads, and other members of the Visuals team.

We're based in Washington, DC. It's cool if you work remotely, but we'll want you here a couple times during the project. (We'd cover those travel costs.)

It's a three-month gig, Februaryish-Aprilish.

Interested? [Email
\[bboyer@npr.org\]\(mailto:bboyer@npr.org\).](mailto:bboyer@npr.org)

Know somebody who'd love this? Please spread the word!!!

This position has been filled. Thanks!

Issue #726: Lazy Imports, Pydantic AI, Classes, and More (March 17, 2026)

PyCoders Weekly

#726 – MARCH 17, 2026 [View in Browser](#) »

The Story of Python's Lazy Imports

This article is about why lazy imports took 3 years and 2 attempts to be added to the language. From PEP 690's rejection to PEP 810's unanimous acceptance. [TECHLIFE](#)

Pydantic AI: Build Type-Safe LLM Agents in Python

Learn how to use Pydantic AI to build type-safe LLM agents in Python with structured outputs, function calling, and dependency injection patterns. [REAL PYTHON](#)

Agents as API Services, Not Prompt Chains.

Clone a working 100-agent Python system: Autonomous Engineering Team, Deep Security Auditor, or Adversarial Code Reviewer. Agents run as API services, discover each other at runtime. No DAGs, no glue code. Any model. Apache 2.0. Clone a Recipe → [AGENTFIELD](#). AI sponsor

When Are Classes Used in Python?

While you don't often need to make your own classes in Python, they can sometimes make your code reusable and easier to read. [TREY HUNNER](#)

Learn the Agentic Coding Workflow That Actually Works on Real Projects

65% of Python developers are stuck using AI for small tasks that fall apart on anything real. This 2-day live course (March 21-22 via Zoom) walks you through building a complete Python CLI app with Claude Code, from an empty directory to a shipped project on GitHub. [REAL PYTHON](#)

DuckDB 1.5.0 Released

[DUCKDB.ORG](#)

PyPy v7.3.21 Released

[PYPY.ORG](#)

Python 3.15.0 Alpha 7

[PYTHON.ORG](#)

Python + AI Content Specialist (Anywhere)

Articles & Tutorials

Crafting and Editing in-Depth Tutorials at Real Python

What goes into creating the tutorials you read at Real Python? What are the steps in the editorial process, and who are the people behind the scenes? This week on the show, Real Python team members Martin Breuss, Brenda

Weleschuk, and Philipp Acsany join us to discuss topic curation, review stages, and quality assurance. [REAL PYTHON](#) podcast

Python loses every public benchmark by 21-875x. Cemrehan took the exact problems people use to dunk on Python and climbed every rung of the optimization ladder: from CPython version upgrades to Rust. Real numbers, real code, real effort costs. [CEMREHAN ÇAVDAR](#)

Ship Voice Agents That Sound Like Human with Async

Async Voice API is a human-like low-latency text-to-speech API for real-time apps and agents. 15 languages, streaming-ready, integrations with n8n, LiveKit, Twilio. Top-ranked on the Hugging Face TTS Arena. From \$0.50/hour with a 24/7 SLA. Try Now → [ASYNC](#) sponsor

Lock the Ghost in uv.lock

In the software world, "remove" is not equal to "gone." Take a short trip through how the Python Package Index handles removals and how you can lock a ghost package in an uv.lock file forever! [CERT. AT](#) • Shared by [Kamil Mańkowski](#)

Comparing PDF Table Extraction Tools

This article explores three Python tools for PDF table extraction: Docling, Marker, and LlamaParse. Learn which handles merged cells and multi-level headers best. [CODECUT. AI](#) • Shared by [Khuyen Tran](#)

What Is Code Review For?

This post explores just what you should and should not use code reviews for. Learn when to use linters to catch problems vs when human review is important. [GLYPH](#)

Caching an Asyncio Function the Easy Way

Caching an async function is trickier than expected, this article walks through why that is and how to use Asyncio primitives to solve the problem. [CHANGS. CO. UK](#) • Shared by [Jamie Chang](#)

Working With APIs in Python: Reading Public Data

Learn how to consume REST APIs with Python using the requests library, including authentication, query parameters, and handling responses. [REAL PYTHON](#) course

Nobody Gets Promoted for Simplicity

Our industry often rewards complexity and ignores simplicity. In interviews, design reviews, and promotions. Here's how to fix it. [TERRIBLE SOFTWARE](#)

You Store Data and You Do Stuff With Data

This post explores the Object Oriented Programming mindset and why you want to associate your data with the operations upon it. [STEPHEN GRUPPETTA](#)

User-generated graphics in the browser with SVG

NPR Apps Blog

The challenge

For NPR's ongoing series "The Changing Lives of Women", we wanted to ask women to share advice gleaned from their experience in the workforce. We've done a few user-generated content projects using Tumblr as a backend, most notably Cook Your Cupboard, so we knew we wanted to reuse that infrastructure. Tumblr provides a very natural format for displaying images as well as baked in tools for sharing and content management. For Cook your Cupboard we had users submit photos, but for this project we couldn't think of a photo to ask our users to take that would say something meaningful about their workplace experience. So, with the help of our friends at Morning Edition, we arrived at the idea of a sign generator and our question:

"What's your note to self – a piece of advice that's helped you at work?"

With that in mind we sketched up a user interface that gave users some ability to customize their submission—font, color, etc—but also guaranteed us a certain amount of visual and thematic consistency.

Making images online

The traditional way of generating images in the browser is to use Flash, which is what sites like quickmeme do. We certainly weren't going to do that. All of our apps must work across all major browsers and on mobile devices. My initial instinct said we could solve this problem with the HTML5 Canvas element. Some folks already use Canvas for resizing images on mobile devices before uploading them, so it seemed like a natural fit. However, in addition to saving the images to Tumblr, we also wanted to generate a very high-resolution version for printing. Generating this on the client would have made for large file sizes at upload time—a deal-breaker for mobile devices. Scaling it up on the server would have lead to poor quality for printing.

After some deliberation I fell upon the idea of using Raphaël.js to generate SVG in the browser. SVG stands for Scalable Vector Graphics, an image format typically used for icons, logos and other graphics that need to be rendered at a variety of sizes. SVG, like HTML, is based on XML and in modern browsers you can embed SVG content directly into your HTML. This also means that you can use standard DOM manipulation tools to modify SVG elements directly in the browser. (And also style them dynamically, as you can see in our recent Arrested Development visualization.)

The first prototype of this strategy came together remarkably quickly. The user selects text, colors and ornamentation. These are rendered as SVG elements directly into the page DOM. Upon hitting submit, we grab the text of the SVG using jQuery's html method and then assign to a hidden input in the form:

The SVG graphic is sent to the server as text via the hidden form field. We've already been running servers for our Tumblr projects to construct the post content and add tags before submitting to Tumblr, so we didn't have to create any new infrastructure for this. (Tumblr also provides a form for having users submit directly, which we are not using for a variety of reasons.) You can see our boilerplate for building projects with Tumblr on the init-tumblr branch of our app-template.

Once the SVG text is on the server we save it to a file and use cairosvg to cut a PNG, which we then POST to Tumblr. Tumblr returns a URL to the new "blog post", which we then send to the user as a 301 redirect. To the user it appears as though they posted their image directly to Tumblr.

Text was probably the hardest thing to get right. Because each browser renders text in a different way we found that our resulting images were inconsistent and often ugly. Worse yet, because our server-side, Cairo-based renderer was also different, we couldn't guarantee the text layout a user saw on their screen would match that of the final image once we'd converted it to a PNG.

Here is the same text (Quicksand 400), rendered in Chrome on the left and IE9 on the right:

Researching a solution for this led me to discover Cufon fonts, a JSON format for representing fonts as SVG paths (technically VML paths, but that doesn't matter). There is a Cufon Javascript library for using these fonts directly, however, there are also built-in hooks for using them Raphaël. (For those who care: they get loaded up via a "magic" callback name.) These resulting fonts are ideal for us, because the paths are already set and thus look the same in every browser and when rendered on the server. It's a beautiful thing:

We found that the various SVG implementations we had to work with (Webkit, IE, Cairo) had different interpretations of width, height and viewBox parameters of the SVG. We ended up using a fixed size for viewBox (2048x2048) and rendering everything in that coordinate reference system. The width and height we scaled with our responsive viewport. On the server width and height were stripped before the SVG was sent to cairosvg, causing it to render the resulting PNGs at viewBox size. See the next section for the code that cleans up the SVG on the server.

Browser support

A similar issue happened with IE9, which for no apparent reason was duplicating the XML namespace attribute of the SVG, xmlns. This caused cairosvg to bomb, so we had to strip it.

Unfortunately, no amount of clever rewriting was ever going to make this work in IE8, which does not support SVG. Note that Raphaël does support IE8, by rendering VML instead of SVG, however, we have no way to get the XML text of the VML from the browser. (And even if we could we

Making Data Tables Responsive

NPR Apps Blog

Left: A data table on a desktop-sized screen. Right: The same table on a small screen, too wide for the viewport.

Data tables with multiple columns are great on desktop screens, but don't work as well at mobile sizes, where the table might be too wide to fit onscreen.

We've been experimenting with a technique we read about from Aaron Gustafson , where the display shifts from a data table to something more row-based at smaller screen widths. Each cell has a data-title attribute with the label for that particular column. On small screens, we:

Set each and to display: block; to make the table cells display in rows instead of columns

Hide the header row

Use :before { content: attr(data-title) “:\00A0”; to display a label in front of each table cell

It works well for simple data tables. More complex presentations, like those involving filtering or sorting, would require more consideration.

Left: A data table on a desktop-sized screen. Right: The same table on a small screen, reformatted for the viewport.

We'll start with some sample data from the Bureau of Labor Statistics that I've dropped into Google Spreadsheets:

Use standard HTML table markup. Wrap your header row in a thead tag — it will be simpler to hide later. And in each td , add a data-title attribute that corresponds to its column label (e.g.,).

Category January February March

Total (16 years and over) 6.6 6.7 6.7

Less than a high school diploma 9.6 9.8 9.6

High school graduates, no college 6.5 6.4 6.3

Some college or associate degree 6.0 6.2 6.1

Bachelor's degree and higher 3.2 3.4 3.4

The CSS

```
body { font: 12px/1.4 Arial, Helvetica, sans-serif; color: #333; margin: 0; padding: 0; }
```

```
table { border-collapse: collapse; padding: 0; margin: 0 0 11px 0; width: 100%; }
```

```
table th { text-align: left; border-bottom: 2px solid #eee; vertical-align: bottom; padding: 0 10px 10px 10px; text-align: right; }
```

```
table td { border-bottom: 1px solid #eee; vertical-align: top; padding: 10px; text-align: right; }
```

```
table th:nth-child(1), table td:nth-child(1) { text-align: left; padding-left: 0; font-weight: bold; }
```

Above, basic CSS styling for the data table, as desktop users would see it.

Below, what the table will look like when it appears in a viewport that is 480px wide or narrower:

```
/* responsive table */ @media screen and (max-width: 480px) { table, tbody { display: block; width: 100%; }
```

Make the table display: block; instead of display: table; and make sure it spans the full width of the content well.

```
thead { display: none; }
```

Hide the header row.

```
table tr, table th, table td { display: block; padding: 0; text-align: left; white-space: normal; }
```

Make all the , and tags display as rows rather than columns. (is probably not necessary to include, since we're hiding the , but I'm doing so for completeness.)

```
table tr { border-bottom: 1px solid #eee; padding-bottom: 11px; margin-bottom: 11px; }
```

Add a dividing line between each row of data.

```
table th[data-title]:before, table td[data-title]:before { content: attr(data-title) “:\00A0”; font-weight: bold; }
```

If a table cell has a data-table attribute, prepend it to the contents of the table cell. (e.g., 6.5 would display as January: 6.5)

```
table td { border: none; margin-bottom: 6px; color: #444; }
```

Table cell style refinements.

```
table td:empty { display: none; }
```

Hide empty table cells.

```
table td:first-child { font-size: 14px; font-weight: bold; margin-bottom: 6px; color: #333; } table td:first-child:before { content: “;” }
```

Make the first table cell appear larger than the others — more like a header — and override the display of the data-title attribute.

```
}
```

And there you go!

Extra: Embed This Table Using Pym.js

Graphics, Photo and Video: Apply for Fall/Winter 2022 Internships at NPR

NPR Apps Blog

NPR's internship program includes several visually-oriented newsroom jobs, working with the News Apps, Visuals and Science teams.

Our paid fall/winter internship program runs from Oct. 3, 2022, to April 15, 2023. In this super-sized internship term, interns may have the opportunity to work with multiple teams in the NPR newsroom and develop a wider range of experience. **DEADLINE EXTENDED:** Applications are due Sunday, July 17, 2022 at 11:59pm ET.

To be eligible, you must be a college student (undergraduate or graduate) or a person who has graduated no more than 12 months prior to the start of the internship period. You must be planning to work from the United States and authorized to work in the United States throughout the internship term.

NewsHub/Graphics Intern (Remote-optional)

For half of the internship, you will be part of the NewsHub team, where you will pitch, report and write stories for NPR's website and learn what it takes to make those stories go big across digital platforms. For the other half of the internship, you will be embedded with the News Apps/Graphics team, focusing on graphics, data visualization and visual storytelling.

This position could be a good fit for someone experienced with graphics and data analysis who wants to improve their reporting skills, or for a reporter with some graphics experience who wants to build stronger graphics/coding chops.

This internship has the flexibility to be on site at our Washington, D. C., office, fully remote, and/or a hybrid version of the two based on the intern's preference.

[Read about our expectations and selection process](#)

APPLY NOW

Multimedia Intern, Visuals and Science Desks (Remote)

This internship is an opportunity to learn more about the world of photo editing. Our goal isn't to make you into a photo editor; we view this internship as a chance for you to understand what it is like to be an editor and improve your visual literacy, which can help you become a better photographer.

On the Visuals Desk, the intern will focus on editing and commissioning visuals for stories for news, national, culture, politics and podcasts. They will also work on content for our other visuals platforms including Instagram. The Visuals Desk intern will work across the newsroom and podcasts. On the Science Desk, interns will edit and commission visuals for stories on global and domestic health, climate change and general science. As one of the largest

desks at NPR, interns will have the opportunity to work with over 30 reporters, producers and editors.

This internship will be fully remote.

APPLY NOW

NPR Music Visuals Intern (Remote)

The NPR Music Video Intern will assist in the production of a range of editorial projects, spanning podcasts, short-form video, Tiny Desk, Alt. Latino and more. We look for video producers who can balance multiple tasks at once, translate big ideas into smart and compelling visuals and who are excited to help NPR Music reach new audiences, especially on platforms like TikTok and Instagram. This is a paid, full-time, 40-hour a week internship working remotely. It will require occasional late nights/odd hours.

APPLY NOW

Dependency Injection with constructors?

mick · Finn.no Tech

The debate whether to use constructors, setters, fields, or interfaces for dependency injection is often heated and opinionated. Should you have a preference?

The argument for Constructor Injection

We had a consultant working with us reminding us to take a preference towards Constructor injection. Indeed we had a large code base using predominantly setter injection because in the past that is what the Spring community recommended.

The arguments for constructor injection goes like:

Dependencies are declared public, providing clarity in the wiring of Dependency Inversion ,

Safe construction , what must be initialised must be called,

Immutability , fields can be declared final, and

Clear indication of complexity through numbers of constructor parameters.

And that Setter injection can be used when needed for cyclic dependencies, optional and re-assignable dependencies, to support multiple/complicated variations of construction, or to free up the constructor for polymorphism purposes.

Being a big fan of Inversion of Control but not overly of Dependency Injection frameworks something smelt wrong to me. Yet solely within the debate of constructor versus setter injection i don't disagree that constructor injection has the advantage. Having been using Spring's dependency injection through annotation a little recently and building a favouritism towards field injection I was happy to get the chance to ponder it over, to learn and to be taught new things. What was it i was missing? Is there a bigger picture?

API vs Implementation

If there is a bigger picture it has to be around the Dependency Inversion argument since this is known to be potentially complex. The point here of using constructor injection is that 1) through a public declaration and injection of dependencies we build an explicit graph showing the dependency inversion throughout the application, and 2) even if the application is wired magically by a framework such injection must still be done in the same way without the framework (eg when writing tests). The latter (2) is interesting in that the requirement on "dependency injection" is too also inverted, that the framework providing dependency injection is removed from the architectural design and becomes solely a implementation detail. But it is the graph in (1) that becomes an important facet in the following analysis.

With this dependency graph in mind what does happen when we bring into the picture a desire to distinguish between API and implementation design...

The DI graph being requested to be clarified by using constructor injection will fall into one of two categories: → 'implementation-specific', an interface defines the public API and the DI is held private by the constructor in the implementation class, → 'API-specific' when the class has no interface. Here everything public is a fully exposed api. There is no implementation-protected visibility here for injectable constructors.

By introducing the constraint of only ever using constructor based injection: in the pursuit of a clarified dependency graph; you remove or make more difficult the ability to publicly distinguish between API and Implementation design.

This distinction between API and implementation is important in being able to create the simple API. The previous blog "using the constretto configuration factory" is a co-incidental example of this. I think the work in Constretto has an excellent implementation design to it, but this particular issue raised frustrations that the API was not as simple as it could have been. Indeed: to obtain the "simplest api"; Constretto (intentionally or not) promotes the use of Spring's injection, a loose coupling that can be compared to reflection. It may be that our usage of Constretto's API, where we wanted to isolate groups of properties, was not what the author originally intended but this only re-enforces the need for designing the simplest possible API.

Therefore it is important to sometimes have all dependency injection completely hidden in the implementation. A clean elegant API must take precedence over a clean elegant implementation. And to achieve this one must first make that distinction between API and Implementation design.

Taking this further we can introduce the distinction between API and SPI . Here a good practice is to stick to using final classes for API and interfaces for SPI. By the same argument as above SPI can't use constructor injection because they don't have constructors.

Inversion-of-Control vs Dependency-Injection

What about the difference between IoC and DI. They are overlapping concepts: the subtlety between the "the contexts" and "the dependencies" rarely emphasised enough. (Java EE 6 has tried to address the distinction between contexts and dependencies at the implementation level with the CDI spec .) The difference between the two, nuanced as it may be, can help illustrate that the DI graph in any application deserves attention in multiple dimensions.

Drawing an application's architecture up as a graph where the vertical axis represents the request stack: that which is typically categorised into architectural layers view, control, and model/services; and the horizontal axis representing the broadness of each architectural layer, then it can be demonstrated that: → IoC generally forms the passing and

Adding a new Bytecode Instruction to the CLR

Matt Warren (.NET)

Now that the CoreCLR is open-source we can do fun things, for instance find out if it's possible to add new IL (Intermediate Language) instruction to the runtime.

TL;DR it turns out that it's easier than you might think!! Here are the steps you need to go through:

Step 0 - Introduction and Background

Step 1 - Add the new IL instruction to the runtime

Step 2 - Make the Interpreter work

Step 3 - Ensure the JIT can recognise the new op-code

Step 4 - Runtime code generation via Reflection. Emit

Step 5 - Future Improvements

Update : turns out that I wasn't the only person to have this idea, see Beachhead implements new opcode on CLR JIT for another implementation by Kouji Matsui .

Step 0

But first a bit of background information. Adding a new IL instruction to the CLR is a pretty rare event, that last time it was done for real was in .NET 2.0 when support for generics was added. This is part of the reason why .NET code had good backwards-compatibility, from Backward compatibility and the .NET Framework 4.5 :

The .NET Framework 4.5 and its point releases (4.5.1, 4.5.2, 4.6, 4.6.1, 4.6.2, and 4.7) are backward-compatible with apps that were built with earlier versions of the .NET Framework. In other words, apps and components built with previous versions will work without modification on the .NET Framework 4.5 .

Side note : The .NET framework did break backwards compatibility when moving from 1.0 to 2.0, precisely so that support for generics could be added deep into the runtime, i.e. with support in the IL. Java took a different decision, I guess because it had been around longer, breaking backwards-comparability was a bigger issue. See the excellent blog post Comparing Java and C# Generics for more info.

Step 1

For this exercise I plan to add a new IL instruction (op-code) to the CoreCLR runtime and because I'm a raving narcissist (not really, see below) I'm going to name it after myself. So let me introduce the matt IL instruction, that you can use like so:

```
.method private hidebysig static int32 TestMattOpCodeMethod(int32 x, int32 y) cil managed noinlining
{
    .maxstack 2 ldarg.0 ldarg.1 matt /\ yay, my name as an IL op-code!!!! ret }
}
```

But because I'm actually a bit-British (i.e. I don't like to 'blow my own trumpet'), I'm going to make the matt op-code almost completely pointless, it's going to do exactly the same thing as calling Math. Max(x, y) , i.e. just return the largest of the 2 numbers.

The other reason for naming it matt is that I'd really like someone to make a version of the C# (Roslyn) compiler that allows you to write code like this:

```
Console.WriteLine ( "{0} m@ {1} = {2}" , 1 , 7 , 1 m @ 7 ); /\
prints '1 m@ 7 = 7'
```

I definitely want the m@ operator to be a thing (pronounced 'matt', not 'm-at'), maybe the other 'Matt Warren' who works at Microsoft on the C# Language Design Team can help out!! Seriously though, if anyone reading this would like to write a similar blog post, showing how you'd add the m@ operator to the Roslyn compiler, please let me know I'd love to read it.

Update : Thanks to Marcin Juraszek (@mmjuraszek) you can now use the m@ in a C# program, see Adding Matt operator to Roslyn - Syntax, Lexer and Parser , Adding Matt operator to Roslyn - Binder and Adding Matt operator to Roslyn - Emitter for the full details.

Now we've defined the op-code, the first step is to ensure that the run-time and tooling can recognise it. In particular we need the IL Assembler (a.k.a ilasm) to be able to take the IL code above (TestMattOpCodeMethod(..)) and produce a .NET executable.

As the .NET runtime source code is nicely structured (+1 to the runtime devs), to make this possible we only need to make changes in opcode.def :

```
— a/src/inc/opcode.def
```

```
+++ b/src/inc/opcode.def
```

```
@@ -154,7 +154,7 @@ OPDEF(CEE_NEWOBJ, "newobj", Var-
Pop, Pu
```

```
OPDEF(CEE_CASTCLASS, "castclass", PopRef, PushRef, In-
lineType, IObjModel, 1, 0xFF, 0x74, NEXT)
```

```
OPDEF(CEE_ISINST, "isinst", PopRef, PushI, InlineType,
IObjModel, 1, 0xFF, 0x75, NEXT)
```

```
OPDEF(CEE_CONV_R_UN, "conv.r.un", Pop1, PushR8, Inli-
neNone, IPrimitive, 1, 0xFF, 0x76, NEXT)
```

```
-OPDEF(CEE_UNUSED58, "unused", Pop0, Push0, Inli-
neNone, IPrimitive, 1, 0xFF, 0x77, NEXT)
```

```
+OPDEF(CEE_MATT, "matt", Pop1+Pop1, Push1, Inli-
neNone, IPrimitive, 1, 0xFF, 0x77, NEXT)
```

```
OPDEF(CEE_UNUSED1, "unused", Pop0, Push0, Inli-
```

Open Source .NET – 1 year later - Now with ASP.NET

Matt Warren (.NET)

In the previous post I looked at the community involvement in the year since Microsoft open-sourced large parts of the .NET framework.

As a follow-up I'm going to repeat that analysis, but this time focussing on the repositories that sit under the ASP. NET umbrella project:

MVC - Model view controller framework for building dynamic web sites with clean separation of concerns, including the merged MVC, Web API, and Web Pages w/ Razor.

DNX - The DNX (a . NET Execution Environment) contains the code required to bootstrap and run an application, including the compilation system, SDK tools, and the native CLR hosts.

EntityFramework - Microsoft's recommended data access technology for new applications in . NET.

KestrelHttpServer - A web server for ASP. NET 5 based on libuv.

In the first part I classified the Issues/PRs as Owner , Col-laborator or Community . However this turned out to have some problems, as was pointed out to me in the comments. There are several people who are non Microsoft employees, but have been made "Collaborators" due to their extensive contributions to a particular repository, for instance @kan-garoo and @benpye .

To address this, I decided to change to just the following 2 categories:

This is possible because (almost) all Microsoft employees have indicated where they work on their GitHub profile, for instance:

There are some notable exceptions, e.g. @shanselman clearly works at Microsoft, but it's easy enough to allow for cases like this.

So after all this analysis, what results did I get. Well overall, the Community involvement accounts for just over 60% over the "Issues Created" and 33% of the "Merged Pull Requests (PRs)". However the amount of PRs is skewed by Entity Framework which has a much higher involvement from Microsoft employees, if this is ignored the Community proportion of PRs increases to 44% .

Issues Created (Nov 2013 - Dec 2015)

Project Microsoft Community Total

aspnet/ MVC 716 1380 2096

aspnet/ dnx 897 1206 2103

aspnet/ EntityFramework 1066 1427 2493

aspnet/ KestrelHttpServer 89 176 265

Total 2768 4189 6957

Merged Pull Requests (Nov 2013 - Dec 2015)

Project Microsoft Community Total

aspnet/ MVC 385 228 613

aspnet/ dnx 406 368 774

aspnet/ EntityFramework 937 225 1162

aspnet/ KestrelHttpServer 69 88 157

Total 1798 909 2706

Note: I included the Kestrel Http Server because it is an interesting case. Currently the #1 contributor is not a Microsoft employee, it is Ben Adams , who is doing a great job of improving the memory usage and in the process helping Kestrel handle more and more requests per/second.

By looking at the results over time, you can see that there is a clear and sustained Community involvement (the lighter section of the bars) over the past 2 years (Nov 2013 - Dec 2015) and it doesn't look like it's going to stop.

Issues Per Month - By Submitter (click for full-size image)

In addition, whilst the Community involvement is easier to see with the Issues per/month, it is still visible in the Merged PRs and again it looks like it has being sustained over the 2 years.

Merged Pull Request Per Month - By Submitter (click for full-size image)

Total Number of People Contributing

It's also interesting to look at the total number of different people who contributed to each project. By doing this you get a real sense of the size of the Community contribution, it's not just a small amount of people doing a lot of work, it's spread across a large amount of people.

This table shows the number of different GitHub users (per project) who opened an Issue or created a PR that was Merged:

Project Microsoft Community Total

aspnet/ MVC 39 395 434

aspnet/ dnx 46 421 467

aspnet/ EntityFramework 31 570 601

aspnet/ KestrelHttpServer 22 95 117

Total 138 1481 1619

FSharp

Clojure Sound - 5 - Double Click with Foot Control

Dragan Djuric

In the last article we created a receiver function that listened to signals from our foot controller and started or stopped playback on consecutive clicks. The trouble with our player is that it only works until the end of the track. The start! function does not automatically rewind the playback. Neither stop! does that. Our program is responsible for detecting that the track should be rewound, perhaps by detecting that we reached the end of track (but even that is not fool proof, since audio infrastructure is not that precise). Now we have to think about a foot user interface that is useful and simple at the same time - there's not many different precise actions that a foot can do.

My idea at this time is the following: distinguish three gestures:

Button (un)pressed (value 0)

Button pressed (value 127)

Button pressed twice in a short time span (doubleclick). This can have two varieties:

First pressed (value 127), then (un)pressed (value 0)

(Un)pressed (value 0), then pressed (value 127).

The values 0 and 127 always interchange, there's no way to send the same value twice in a row (with the MIDI controller I have).

This gives us 4 different signals from one button that we can work with, which doesn't seem that much, but on the other hand, we can't assume that the feet of our user that is doing all this stomping is able or eager for much more complicated stuff.

So, if I assume that the particular button is dedicated to a particular clip playback, I see the following actions:

If the button is (un)pressed (value 0), the clip should stop, regardless of the previous state. If the clip has not been playing, nothing changes.

If the button is pressed (value 127), the clip should start, regardless of the previous state. If the clip has been playing, it just keeps playing.

If the button is clicked twice in a row, the things might get complicated!

It's not exactly rocket science level complicated, but we still need to think what to do. In our desktop user interface toolkits, we are accustomed to working with a very high level API. We write functions that react to gestures (click, left click, right click, double click, wheel scroll, touch, etc.) but we are not responsible for detecting these gestures from the signals that our Magic Trackpad sends; the drivers and

the operating system take care of that complexity. We just write actions that happen on Doubleclick.

But here, we have to take care of the whole stream of raw signals. How should we distinguish between an action that happens when the button is clicked once, and an action that should register that first click, and wait for the second one, which may never come? And, we would like the solution to be simple!

I drew a simple state transition diagram (with a pen on a napkin) and concluded that for our limited foot control it can be simplified to this:

It all depends whether the clip position was 0, maximum value, or something in between.

Dragan Djuric

As soon as the button is pressed, evaluate either start! (value 127), or stop! (value 0) function.

When the button is pressed shortly after it has been pressed the previous time (say, 600 milliseconds), treat it as a double click, and initiate the alternative action.

This leaves us with two tasks:

Detecting doubleclick

Deciding on a simple decision process for the alternative action (remember, the user should anticipate the action while playing the guitar).

The usual imports:

```
(ns my-midi) (require '[uncomplicate.commons.core :refer [close! info]] '[uncomplicate.clojure-sound [core :refer :all]] [midi :refer :all] [sampled :refer :all]))
```

Detecting doubleclick

If you went back to one of the previous articles and connect the println as a receiver for the controller you have, you'd see that each message contains its timestamps in microseconds. We can detect whether a "click" message comes shortly after the previous "unclick" simply by subtracting the previous message timestamp. The only trouble is that we don't have access to message history.

If we saved the last message, we have all the information we need; not only that we know both timestamps, we can even recognize whether the last message is of the same kind, and coming from the same button! Fast pressing different buttons does not count as "doubleclick". Clojure gives us several elegant facilities for state management. In this case, the state is internal, so we don't have to care about synchronization. We don't need what refs, agents, or atoms offer. I think that the simple volatile! can serve our needs rather well.

The great big Schibsted programming language survey 2015

Morten Lied Johansen · Finn.no Tech

During September, I collected answers on a survey about programming languages from people who work in one of the many companies that make up the Schibsted family. The survey is neither requested, sanctioned, approved or even suggested by anyone in management, in any Schibsted company. It is entirely my own personal pet-project, because I'm interested in these things.

The survey opens with the following introduction:

In 2012, there was a discussion about technology choices, and particularly programming languages to use in FINN. As always in such discussions, claims like "It's impossible to hire people who know language X, so we need to always use Java" or "We can't expect programmers to learn a new language just because we think language Y would be a better fit" were put forward.

Personally, I've never liked those arguments, because they contradict what I think defines a "good programmer", and I'm optimistically hoping that we only hire "good programmers". Also, instead of opinions, beliefs, rumours and hearsay, let's be data driven in these discussions! So I decided I wanted to put it to the test, and created a survey and posted it for all the company to see.

The results from that survey were blogged about here: <http://tech.finn.no/2012/09/04/leaving-the-tower-of-babel/>

Now, almost three years later, a lot of changes are happening. FINN is becoming more close with it's Schibsted brethren, Schibsted itself is turning towards a more technological outlook, and I felt it was time to get an updated view, and why not involve all of Schibsted while we're at it.

So, "The great big Schibsted programming language survey 2015" was created, and hopefully it will make the rounds in all of Schibsted and gather some interesting, fun, surprising and (with a bit of luck) useful insights.

About the responses

309 people responded to the survey, which is more than many of my co-workers thought would respond.

Over 50% of respondents are in their thirties, and if we look at the ages from 26 to 40, we cover over 75% of respondents. On gender, it's even worse, with a whopping 93.5% of respondents admitting to being male. We also seem to shun inexperienced people, with 42% having 10 years of experience or more, and only 2.3% of respondents fresh from school.

It seems we have some work to do with regards to diversity.

According to Arena (The Schibsted-wide intranet), there are just under 125 companies in the Schibsted familiy. In those companies, 9355 people are employed.

That means somewhere between 20% and 30% response rate in the target group, which isn't too bad.

Unfortunately, our results are not representative. FINN is by no means the largest company, but supplied the most answers with 80 respondents. The second largest group was the various incarnations of Schibsted Product & Technology (SPT), which supplied 53 responses if we combine them all. Those two supplied over a third of all responses. Third place belongs to Schibsted Tech Polska (16 responses), which beat Wilhaben (15), Le Bon Coin (13), Blocket (12) and Aftonbladet (10) to close out the top 7.

This probably means that the results are more a description of FINN and SPT than a description of Schibsted in general.

Those 309 people listed 103 different companies, which is about 20 less than the total number of companies. That sounds great! Except I'm fairly certain that's not what I have. Trying to normalize the data entered here, fixing variations in spelling and other issues I'll get back to in a minute, I reduced it to 45 companies, one of which was Schibsted itself.

Those issues are companies that don't exist (atleast according to Arena)! I've tried my best to guess which company was really intended, but might have made mistakes here and there. In some cases, I had no idea which company to use instead, so I just left it as is, even if the company is not listed as part of Schibsted. Some examples: Schibsted Tech Madrid, SPT - Madrid, SCM Spain, SCM Central, Schibsted Norge Tech, Plan3 and Schibsted ATE. None of these were listed anywhere, but I've tried my best to place them some logical place.

Another problem was SPT aka Schibsted Products & Technology. SPT is techically four companies, SPT Norway, SPT Sweden, SPT Spain and SPT UK. Most people in SPT simply answered SPT, but they estimated as little as 20 people working in SPT, and as high as 250. I've disregarded their disagreement, and combined them all to just SPT.

Two people said they work in Schibsted, but they had wildly differing opinions about how many people work in Schibsted. One said 10000 total, 1500 technical, the other said 1000 total, 200 technical.

Disregarding the problems here, let's look at the numbers entered. Using the averages for each company and summing up, we arrive at roughly 4500 people working in those 47 companies. For technical positions, the sum of averages is roughly 1600. Since we have responses from about half the companies, that fits nicely with the total being just under 10000.

Languages in regular use

The most commonly used language, if you can call it that, is Bash. 52.1% of respondents use Bash regularly. Close behind, at 50.2% is JavaScript. Neither of these are surprising, considering the business we're in. Java clocks in at 49.2%,

Adding a Spell Checker to a Jekyll Blog

Daniel Doubrovkine · Daniel Doubrovkine (dBlock)

I found it annoyingly non-trivial to add a spell checker to this blog.

For now, I settled on GitHub Spellcheck Action that uses PySpelling on files changed in the commit or pull request as described in this blog post .

```
name : Check Spelling on :
[ push , pull_request ] jobs :
check : runs-on : ubuntu-latest
steps :
```

- uses : actions/check-out@v3
- uses : tj-actions/changed-files@v46.0.4

```
id : changed_files with :
files : | **/*.md **/*.markdown
```

- name : Check Spelling

```
uses : rojopolis/spellcheck-github-actions@0.45.0
```

```
with : task_name : Markdown config_path : .pyspelling.yml
```

```
source_files : ${{ steps.changed_files.outputs.all_changed_files }}
```

To run PySpelling locally ensure you have a working version of Python, install PySpelling with pip install pyspelling , and aspell with brew install aspell on a Mac. In my configuration I also use pymdownx from pymdown-extensions which is installed with pip install pymdown-extensions .

You need a .pyspelling.yml and you can run it as follows.

```
pyspelling -
config .pyspelling.yml
```

This is a Jekyll blog in which we want to ignore code, wrapped between Jekyll magic commands for syntax highlighting. This can be accomplished with a PySpelling pipeline in the

above-mentioned configuration file.

pipeline :

- pyspelling.filters.context :

```
context_visible_first : true
delimiters : # ignore jekyll multiline magic highlights
{% ... %}
```

- open : ‘ (? s) ^ { \ % highlight . * \ % \ } \$ ’

```
close : ‘ ^ { \ % endhighlight \ % \ } $ ’ # ignore the rest of jekyll magic commands
```

- open : ‘ { % ’

```
close : ‘ % ’
```

Finally, we can collect the initial set of words to potentially exclude from existing posts into .pyspelling.words .

The file helped me spot a few spelling mistakes, now fixed. See code.dblock.org/#134 for the full change.

Update: I eventually switched to using regular backticks for code when adding a style checker in #136 , so the above delimiters rule changed as follows.

- pyspelling.filters.context :

```
context_visible_first : true
escapes : \\ [ \ ` ] delimiters :
• open : “ ( ? s ) ^ ( ? P * ` { 3 , } ) ( [ a - z + ] * ) $ ”
```

```
close : “ ^ ( ? P = open ) $ ”
```

- open : “ (? P ` +) ”

```
close : “ ( ? P = open ) ”
```

- open : ‘ { % ’

```
close : ‘ % ’
```

Adding a Spell Checker to a Jekyll Blog was originally published by Daniel Doubrovkine at code.dblock.org | tech blog on December 30, 2024.

Work In Public! (Or, why you really should consider being NPR’s Knight-Mozilla fellow!)

NPR Apps Blog

Visual journalism experts. David Sweeney/NPR.

It’s joy to work in public media.

Folks here do amazing journalism, and are awesome to work with.

Why? The non-commercial relationship between us and our audience. We’re not selling them anything. We do sell sponsorship, but have you heard an ad on NPR? They’re the nicest, dumbest ads you’ve ever heard, and they aren’t our primary source of income. No, public media exists because, for more than 40 years, our audience has sent us money, just because they want us to keep up the good work.

And this relationship, built with love and trust, permeates the newsroom and the whole organization. It’s fucking cool.

Why Visuals?

The visuals team is trying something weird. We’re a small team (a dozen, not including interns), and we handle all aspects of visual storytelling at NPR. We make and edit: charts and maps , data visualizations , photography and video , and lots of experimental , web-native stories .

We’re mission-driven , and believe that being open-source and transparent in our methods is essential to our role as public media.

And we’re having a pretty fun time doing it.

You.

A fellow on our team will not get a special project, just for you. We don’t work that way . You’ll be our teammate: making stuff with us, learning what we’ve learned, teaching us what you know and what you’re learning elsewhere during your fellowship year.

Your perspective is even more valuable than your skills. We’re still just figuring out the best ways for a mashed-up visual journalism team to work together and make great internet. You will help us make this thing happen.

So come along for the ride! I guarantee it’ll be a tremendous year. Apply now! (It closes August 16th!)

class="highlight"> pipeline : - pyspelling.

Daniel Doubrovkine

CodeSOD: Awaiting A Reaction

Remy Porter · The Daily WTF

Today's Anonymous submitter sends us some React code. We'll look at the code and then talk about the WTF:

```
// inside a function for
updating checkboxes on
a page if (e.target .
checked) { const removeIndex = await checkedlist.
findIndex ( ( sel ) =>
sel. Id == selected. Id , )
const removeRowIndex =
await RowValue . findIndex
( ( sel ) => sel == Index , )
```

```
// checkedlist and Row-
Value are both useState
instances.... they should
never be modified directly
await checkedlist. splice
(removeIndex, 1 ) await
RowValue . splice (re-
moveRowIndex, 1 )
```

```
// so instead of doing
above logic in the set state,
they dont setCheckedlist
(checkedlist) setRow ( Row-
Value ) } else { if
(checkedlist. findIndex
( ( sel ) => sel. Id ==
selected. Id ) == - 1 )
{ await checkedlist. push
(selected) } // same, in-
stead of just doing a
set state call, we do
awaits and self updates
await RowValue . push
( Index ) setCheckedlist
(checkedlist) setRow ( Row-
Value ) }
```

Comments were added by our submitter.

This code works. It's the wrong approach for doing things in React: modifying objects controlled by react, instead of using the provided methods, it's doing asynchronous push calls. Without the broader context, it's hard to point out all the other ways to do this,

but honestly, that's not the interesting part.

I'll let our submitter explain:

This code is black magic, because if I update it, it breaks everything. Somehow, this is working in perfect tandem with the rest of the horrible page, but if I clean it up, it breaks the checkboxes; they're no longer able to be clicked. Its forcing React somehow to update asynchronously so it can use these updated values correctly, but thats the neat part, they aren't even being used anywhere else, but somehow the re-rendering page only accepts awaits. I've tried refactoring it 5 different ways to no avail

That's what makes truly bad code. Code so bad that you can't even fix it without breaking a thousand other things. Code that you have to carefully, slowly, pick through and gently refactor, discovering all sorts of random side-effects that are hidden. The code so bad that you actually have to live with it, at least for awhile.

[Advertisement] BuildMaster allows you to create a self-service release management platform that allows different teams to manage their applications. Explore how!

Be our design/code/??? intern for winter/spring 2017!

NPR Apps Blog

Map by Visuals Team intern Brittany Mayes

Are you data-curious, internet savvy, and interested in journalism? Do you draw, design, or write code? We are looking for you.

We've had journalists who are learning to code, programmers who are learning about journalism, designers who love data graphics, designers who love UX, reporters who love data, and illustrators who make beautiful things.

Does this sound like you? Please join our team! It isn't always easy, but it is very rewarding. You'll learn a ton and you'll have a lot of fun.

The internship runs from January 9, 2017 to April 21, 2017. Applications are due Sunday, November 6, 2016 at 11:59pm eastern .

Here's how to apply

Read about our expectations and selection process and then apply now!

Into images? Check out our photo editing internship .

Putting a mustache on Tiles-3

mick · Finn.no Tech

We're proud to see a contribution from one of our developers end up in the Tiles-3 release!

The front-end architecture of FINN.no is evolving to be a lot more advanced and a lot more work is being done by client-side scripts. In order to maintain first time rendering speeds and to prevent duplicating template-code we needed something which allowed us to reuse templates both client- and server-side. This is where mustache templates have come into play. We could've gone ahead and done a large template framework review, like others have done, but we instead opted to just solve the problem with the technology we already had.

Morten Lied Johansen's contribution allows Tiles-3 to render mustache templates. Existing jsp templates can be rewritten into mustache without having to touch surrounding templates or code!

The code please

To get Tiles-3 to do this include the tiles-request-mustache library and configure your TilesContainerFactory like

```
protected void registerAttributeRenderers(...)
{ MustacheRenderer mustacheRenderer = new
MustacheRenderer(); mustacheRenderer.setAcceptPattern(Pattern.compile(".*mustache.*"));
rendererFactory.registerRenderer("mustache", mustacheRenderer); ... }
protected Renderer createTemplateAttributeRenderer(...) { final ChainedDelegateRenderer chainedRenderer =
```

```
new ChainedDelegateRenderer(); chainedRenderer.addAttributeRenderer("mustache"); ... }
```

then you're free to replace existing tiles attributes like

with stuff like

Good stuff FINN!

Be our design/dev intern for fall 2019!

NPR Apps Blog

Are you data-curious, interested in journalism? Do you design or write code? We are looking for you.

We've had journalists who are learning to code, programmers who are learning about journalism, designers who love data graphics and reporters who love data.

Does this sound like you? Apply to join our team! It isn't always easy, but it is very rewarding. You'll learn a ton and you'll have a lot of fun.

Our paid internship runs from Sept. 9 to Dec. 13. DEADLINE EXTENDED: Applications are due Friday, July 19 at 5 p.m. ET.

Here are a few projects our recent interns have worked on:

Vanessa Qian (Summer 2018)

Jasmine Mithani (Spring 2018)

Matthew Zhang (Fall 2017)

Leanne Abraham (Summer 2017)

Eunice Esomonu (Spring 2017)

Clinton King (Fall 2016)

Brittany Mayes (Summer 2016)

Who's Eligible

To be eligible for an internship with NPR, you must be a student (undergraduate or graduate) or a person who has graduated no more than 12 months prior to the start of the internship period. You must also be

authorized to work in the United States ("mustache")

Here's how to apply

Read about our expectations and selection process and then apply now! (The Fall 2019 application period has ended.)

Into photography? Check out our photo editing internship .

Morten Lied Johansen’s contribution allows Tiles-3 to render mustache templates.

mick

BRIEFS

IN BRIEF

Nicole Samoroukova, The Reformed Broker (Josh Brown): Join Downtown Josh Brown and Michael Batnick for another round of What Are Your Thoughts? On this week’s episode, Josh and Michael discuss the biggest topics in investing and finance, including: ►Sam Altman – OpenAI turmoil. ►Buyback SURGE – “\$QQQ largest weekly inflows on record this past week.” ►NVDA Earnings – “The chip maker’s earnings beat Wall Street expectations in...

The post All Hell Breaks Loose in AI appeared first on The Reformed Broker .

Evan Miller, Evan Miller: A/B tests run faster with CUPED. Here I explain the underlying math, and use it to invent an even better variance-reduction technique for online experiments: You Can’t Spell CUPED Without Frisch-Waugh-Lovell

John Noonan, Redis Blog: AI systems don’t fail on a schedule. They drift in production while you’re busy writing the quarterly report or firefighting an unrelated outage—and that drift creates exposure. Third-party breaches have doubled from 15% to 30% in just one year, with ...

Jim Allen Wallace, Redis Blog: A user asks your support chatbot “how do I reset my password?” and it pulls the right help doc, generates a clear answer, and responds in under a second. The next user asks “what’s your refund policy?” and the chatbot retrieves three barely relevant p...

The Silicon Ledger

Vol. 1, No. 090 · 2026-03-30

Curated and composed automatically.